actuarial-science: the Actuarial Science and Insurance program — user guide

actuarial-science-ai999x 1.1.0, Rust build: command line, desktop app and browser. **Creator / engineer:** Ibnul Jaif Farabi · **Company:** AI999X · support@ai999x.com

Actuarial Science and Insurance covers life tables and survival models, life insurances and annuities, net and gross premiums, reserves and Thiele's equation, interest theory, loss models with deductibles and limits, Panjer's recursion, VaR and TVaR, credibility and loss reserving (chain-ladder, Bornhuetter-Ferguson, Cape Cod, Mack). This guide is about `actuarial-science`, the model as a self-contained program for Windows and Linux: 91 capabilities in 15 topics, answered on your own computer with no network, no Python and nothing else to install. The same compiled model runs in the AI999X desktop app and in the browser on the model's page (https://www.ai999x.com/models/actuarial-science-ai999x/); the Python package (`pip install actuarial-science-ai999x`) has its own guide. Every command output printed here is what the program printed.

Contents

- 1. Get it: desktop app, command line, browser, Python
- 2. Quick start
- 3. Writing inputs
- 4. Outcomes and exit codes
- 5. Scripts and agents: batch and JSON
- 6. Figures
- 7. In a web page or Node.js: WebAssembly
- 8. Every capability

1. Get it

Version 1.4.0 of the AI999X desktop app and of the command-line bundle. Checksums for every file: https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS; the newest version is always under https://www.ai999x.com/downloads/desktop/latest/. Full instructions: https://www.ai999x.com/desktop-cli/

what	system	download
Desktop app: Actuarial Science and Insurance	Windows x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-actuarial-science-1.4.0-windows-x86_64.zip
Desktop app: Actuarial Science and Insurance	Linux x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-actuarial-science-1.4.0-linux-x86_64.tar.gz
Desktop app: all models	Windows x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-1.4.0-windows-x86_64.zip
Desktop app: all models	Linux x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
Command line	Windows x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-cli-1.4.0-windows-x86_64.zip

what	system	download
Command line	Linux x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz

Desktop app

Actuarial Science and Insurance in its own app: search, questions in words, input forms, live figures and explanations. Windows: Download ai999x-actuarial-science-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x-actuarial-science.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-actuarial-science-1.4.0-linux-x86_64.tar.gz
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-actuarial-science-1.4.0-linux-x86_64.tar.gz
./ai999x-actuarial-science-1.4.0-linux-x86_64/ai999x-actuarial-science
```

It opens on this model: type a question or pick a capability, fill in the form and press Compute. The AI999X desktop app with every model works the same way (choose the model first). Windows: Download ai999x-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-1.4.0-linux-x86_64.tar.gz
./ai999x-1.4.0-linux-x86_64/ai999x
```

Command line — Linux

The bundle holds one program per model; `actuarial-science` is this one. The checksum line should print OK.

```
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
mkdir -p ~/.local/share && tar xzf ai999x-cli-1.4.0-linux-x86_64.tar.gz -C ~/.local/share
echo 'export PATH="$HOME/.local/share/ai999x-cli-1.4.0-linux-x86_64:$PATH"' >> ~/.bashrc && source ~/.bashrc
actuarial-science --version
```

Command line — Windows (PowerShell)

The fourth line should print True.

```
$F = "ai999x-cli-1.4.0-windows-x86_64.zip"
Invoke-WebRequest "https://www.ai999x.com/downloads/desktop/1.4.0/$F" -OutFile $F
Invoke-WebRequest "https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS" -OutFile SHA256SUMS
(Get-FileHash $F -Algorithm SHA256).Hash.ToLower() -eq ((Select-String -SimpleMatch $F
SHA256SUMS).Line -split ' ')[0]
Expand-Archive $F -DestinationPath "$env:LOCALAPPDATA\AI999X" -Force
$D = "$env:LOCALAPPDATA\AI999X\ai999x-cli-1.4.0-windows-x86_64"; $env:Path += ";$D"
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path", "User")
+ ";$D", "User")
actuarial-science --version
```

In your browser

Open https://www.ai999x.com/models/actuarial-science-ai999x/, choose a capability and press Compute in your browser: the same program runs on your device as WebAssembly; nothing is sent anywhere.

Python

```
pip install actuarial-science-ai999x
```

2. Quick start

2.1 Check the install

Prints the model, its version and that this is the Rust build.

```
$ actuarial-science --version
actuarial-science-ai999x 1.0.0 (Rust build, ax-engine 0.1.0)
```

2.2 Find a capability

Search the capabilities by what you want to know; the first column is the id to pass to info and solve.

```
$ actuarial-science search "net annual premium for whole life insurance"
act_net_premium_whole_life           Net annual premium for whole life insurance  (premiums / net
premiums)
act_net_premium_limited_pay          Net annual premium for limited-payment whole life insurance
(premiums / net premiums)
act_reserve_whole_life               Net premium reserve for whole life insurance  (reserves /
prospective reserves)
act_gross_premium                    Gross annual premium for whole life insurance with expenses
(premiums / gross premiums)
act_makeham_continuous_whole_life    Continuous whole life insurance, premium and reserve under
Makeham's law  (reserves / Thiele's differential equation)
act_net_premium_term                 Net annual premium for term insurance  (premiums / net
premiums)
act_net_premium_endowment            Net annual premium for endowment insurance  (premiums / net
premiums)
act_gross_premium_reserve            Gross premium reserve for whole life insurance with expenses
(reserves / gross premium reserves)
… 12 more
```

2.3 List the capabilities

Every capability the program answers: its id (what you pass to solve) and its name.

```
$ actuarial-science list
act_accumulated_value                    Accumulated and present value of a single amount
act_annuity_certain                      Annuity-certain present and accumulated values
act_annuity_yield                        Yield rate of a level annuity
act_bornhuetter_ferguson                 Bornhuetter-Ferguson reserve
act_buhlmann_credibility                 Buhlmann credibility premium (known structural parameters)
act_buhlmann_empirical                   Empirical Buhlmann credibility from a data table
act_buhlmann_straub                      Buhlmann-Straub credibility with exposures
act_cape_cod                             Cape Cod (Stanard-Buhlmann) reserve
… 83 more
```

2.4 What a capability needs

The equation or method, every input and output with its SI unit. Read this before solve: it tells you the input names.

```
$ actuarial-science info act_net_premium_whole_life
Net annual premium for whole life insurance
  equivalence principle: P ä_x = S A_x
  in   q        death probabilities q_x, q_(x+1), ... ending with 1 (a list) (vector) []
  in   i        annual effective interest rate (5% or 0.05) []
  in   S        sum insured []
  out  P        annual net premium S A_x / ä_x []
  out  A        whole life insurance A_x []
  out  a_due    whole life annuity-due ä_x []
```

2.5 Solve

Inputs as name=value, each with any unit of the right kind. The answer lists every output, the method used, each input converted to SI, and the conditions the answer assumes.

```
$ actuarial-science solve act_net_premium_whole_life "q=[0.1, 0.2, 0.5, 1]" i=0.05 S=1000.0
P = 306.8418549
A = 0.8656578278
a_due = 2.821185617
  by: Net annual premium for whole life insurance  (equivalence principle: P ä_x = S A_x)
  i = 0.05
  S = 1000
  assumes: the table closes; 0 < i < 1; S > 0
```

2.6 Machine-readable answer

--json prints the same answer as one line of JSON (values in SI): for scripts and other programs.

```
$ actuarial-science solve act_net_premium_whole_life "q=[0.1, 0.2, 0.5, 1]" i=0.05 S=1000.0 --json
{"outcome":"SOLVED","message":"","value":306.841854896,"unit":"","solved_for":"P","si":[["i",0.05],
["S",1000.0]],"values":[["P",306.841854896,""],["A",0.865657827757,""],
["a_due",2.8211856171,""]],"target":"P","phrase":"annual net premium S A_x / ä_x","assumptions":
["the table closes","0 < i < 1","S > 0"],"name":"Net annual premium for whole life
insurance","equation":"equivalence principle: P ä_x = S A_x"}
```

2.7 When an input is missing

No number is guessed: the answer names what is missing, shows a complete call, and the program exits with status 1.

```
$ actuarial-science solve act_net_premium_whole_life "q=[0.1, 0.2, 0.5, 1]" i=0.05
NEEDS_INPUT: act_net_premium_whole_life needs S (sum insured). Example:
solve('act_net_premium_whole_life', q=[0.1, 0.2, 0.5, 1.0], i=0.05, S=1000).
```

2.8 Draw the figures

The live figures for your own inputs, written as SVG files into the folder given with --out (here the current folder); open them in any browser. Without --out the SVG text comes back as JSON.

```
$ actuarial-science figures act_net_premium_whole_life "q=[0.1, 0.2, 0.5, 1]" i=0.05 S=1000.0 --out
.
[
  {
    "slug": "act-net-premium-whole-life-field",
    "title": "Annual net premium S A_x / ä_x over S and i",
    "caption": "P (annual net premium S A_x / ä_x) over S from 500 to 1500 and i from 0.005 to 0.5
(logarithmic vertical axis): each of the 221 cells is one run of the model, shaded by its value
(light 80.04 to dark 498.8); refused combinations are left blank. Marked: these inputs, S = 1000, i
= 0.05, P = 306.8. Your inputs: q = (list), i = 0.05, S = 1000.0.",
    "file": ".\\act-net-premium-whole-life-field.svg"
  },
  {
    "slug": "act-net-premium-whole-life-live-1",
    "title": "Annual net premium S A_x / ä_x as sum insured varies",
    "caption": "Annual net premium S A_x / ä_x as S runs from 250 to 1750, every other input as you
entered it (41 solutions). Marked: your S = 1000 gives P = 306.8. Your inputs: q = (list), i =
0.05, S = 1000.0.",
    "file": ".\\act-net-premium-whole-life-live-1.svg"
  },
  {
    "slug": "act-net-premium-whole-life-live-2",
    "title": "Annual net premium S A_x / ä_x as annual effective interest rate (5% or 0.05)
varies",
    "caption": "Annual net premium S A_x / ä_x as i runs from 0.0125 to 0.0875, every other input
as you entered it (29 solutions). Marked: your i = 0.05 gives P = 306.8. Your inputs: q = (list), i
= 0.05, S = 1000.0.",
    "file": ".\\act-net-premium-whole-life-live-2.svg"
  }
]
```

2.9 Many questions at once

One JSON request per line in, one JSON answer per line out, in order: for scripts, notebooks and agents (the program starts once). In PowerShell: Get-Content cases.jsonl | actuarial-science batch

`cases.jsonl`:

```
{"key": "act_net_premium_whole_life", "inputs": {"q": "[0.1, 0.2, 0.5, 1]", "i": 0.05, "S":
1000.0}}
{"key": "act_annuity_certain", "inputs": {"n": 10, "i": 0.05}}
```

```
$ actuarial-science batch < cases.jsonl
{"outcome":"SOLVED","message":"","value":306.841854896,"unit":"","solved_for":"P","si":[["i",0.05],
["S",1000.0]],"values":[["P",306.841854896,""],["A",0.865657827757,""],
["a_due",2.8211856171,""]],"target":"P","phrase":"annual net premium S A_x / ä_x","assumptions":
["the table closes","0 < i < 1","S > 0"],"name":"Net annual premium for whole life
insurance","equation":"equivalence principle: P ä_x = S A_x"}
{"outcome":"SOLVED","message":"","value":7.72173492918,"unit":"","solved_for":"a_n","si":
[["i",0.05]],"values":[["a_n",7.72173492918,""],["a_due",8.10782167564,""],
["s_n",12.5778925355,""],["s_due",13.2067871623,""],
["a_cont",7.91320859505,""]],"target":"a_n","phrase":"present value of an annuity-immediate
a_n","assumptions":["n >= 1","0 < i < 1"],"name":"Annuity-certain present and accumulated
values","equation":"a_n = (1 - v^n)/i, a-due = (1 - v^n)/d, s_n = ((1+i)^n - 1)/i, s-due = ((1+i)^n
- 1)/d, continuous = (1 - v^n)/delta"}
```

3. Writing inputs

Inputs are written `name=value`, one per argument; `actuarial-science info <capability>` lists the names. Quote an argument that holds a space or brackets (bash, zsh and PowerShell all accept double quotes). The answer always states each input as it was read, converted to SI, so a misread unit is visible at once.

kind	example	how it is read
a number with a unit	`"R=0.128 nm"`	any unit of the same kind; converted to SI
a number in SI	`R=1.28e-10`	no unit means the SI unit shown by info
a choice	`structure=FCC`	one of the values info lists
a list or vector	`"v=[1, 2, 3]"` or `"v=1, 2, 3"`	brackets keep it a list
a matrix	`"A=[[1, 2], [3, 4]]"`	one list per row
true / false	`inclusive=true`	for yes/no options

An equation can be solved for any of its quantities: give the others and add `--target <symbol>`. Without `--target`, the program solves for the quantity the capability is named after.

4. Outcomes and exit codes

The program never guesses a number: when it cannot answer, it says why and exits with status 1, so a script can tell an answer from a refusal without reading the text.

outcome	meaning	exit status
SOLVED	the answer: every output with its unit, the method, the inputs in SI, the conditions assumed	0

outcome	meaning	exit status
NEEDS_INPUT	an input is missing; the message names it and shows a complete call	1
INVALID_INPUT	a value or unit of the wrong kind, an unknown input name, or a value outside the conditions	1
AMBIGUOUS	the inputs allow two different answers; say which one is meant	1
OUT_OF_SCOPE	the model has no capability for this, or the result would not be meaningful	1
(usage)	a command the program does not know: it prints its usage	2

5. Scripts and agents: batch and JSON

`actuarial-science solve ... --json` prints one answer as one line of JSON. `actuarial-science batch` reads one request per line on standard input and writes one JSON answer per line, in order, starting the program once for any number of requests. A request is `{"key": "<capability>", "inputs": {"name": "value", ...}, "target": null}`. In PowerShell, pipe the file in: `Get-Content cases.jsonl | actuarial-science batch`. `actuarial-science catalog` prints every capability with its inputs as JSON, to build forms or tool definitions.

answer field	meaning
outcome	one of the outcomes above
message	why there is no answer (empty when SOLVED)
value, unit	the main result in SI units
values	every output: [symbol, value, unit] (procedures)
target	the quantity solved for
si	each numeric input converted to SI: [symbol, value]
assumptions	the conditions the answer relies on
name, equation	the capability and its equation or method

```
import json, subprocess

reqs = [{"key": "<capability>", "inputs": {"name": "value"}}]
out = subprocess.run(["actuarial-science", "batch"], input="\n".join(map(json.dumps, reqs)),
                     capture_output=True, text=True, check=True).stdout
answers = [json.loads(line) for line in out.splitlines()]
```

6. Figures

`actuarial-science figures <capability> name=value ... --out DIR` draws the live figures for those inputs and writes each as an SVG file into DIR (an existing folder); without `--out` the SVG text comes back as JSON. The desktop app and the model page draw the same figures as soon as you press Compute.

```
$ actuarial-science figures act_net_premium_whole_life "q=[0.1, 0.2, 0.5, 1]" i=0.05 S=1000.0 --out
.
[
  {
    "slug": "act-net-premium-whole-life-field",
    "title": "Annual net premium S A_x / ä_x over S and i",
    "caption": "P (annual net premium S A_x / ä_x) over S from 500 to 1500 and i from 0.005 to 0.5
(logarithmic vertical axis): each of the 221 cells is one run of the model, shaded by its value
(light 80.04 to dark 498.8); refused combinations are left blank. Marked: these inputs, S = 1000, i
= 0.05, P = 306.8. Your inputs: q = (list), i = 0.05, S = 1000.0.",
    "file": ".\\act-net-premium-whole-life-field.svg"
  },
  {
    "slug": "act-net-premium-whole-life-live-1",
    "title": "Annual net premium S A_x / ä_x as sum insured varies",
    "caption": "Annual net premium S A_x / ä_x as S runs from 250 to 1750, every other input as you
entered it (41 solutions). Marked: your S = 1000 gives P = 306.8. Your inputs: q = (list), i =
0.05, S = 1000.0.",
    "file": ".\\act-net-premium-whole-life-live-1.svg"
  },
  {
    "slug": "act-net-premium-whole-life-live-2",
    "title": "Annual net premium S A_x / ä_x as annual effective interest rate (5% or 0.05)
varies",
    "caption": "Annual net premium S A_x / ä_x as i runs from 0.0125 to 0.0875, every other input
as you entered it (29 solutions). Marked: your i = 0.05 gives P = 306.8. Your inputs: q = (list), i
= 0.05, S = 1000.0.",
    "file": ".\\act-net-premium-whole-life-live-2.svg"
  }
]
```

7. In a web page or Node.js: WebAssembly

The same model is a WebAssembly module, `actuarial_science_ai999x.wasm` (https://www.ai999x.com/compute/rust/actuarial_science_ai999x.wasm), with a small loader, `ax-wasm.js` (https://www.ai999x.com/compute/rust/ax-wasm.js). It runs in a browser, a Web Worker or Node.js, with no server.

```
<script src="ax-wasm.js"></script>
<script>
  AXWasm.load("actuarial_science_ai999x.wasm").then(function (m) {
    var a = m.solve({key: "<capability>", inputs: {name: "value"}});
    console.log(a.outcome, a.values || a.value);
    console.log(m.catalog().capabilities.length + " capabilities");
  });
</script>
```

8. Every capability

By topic. For each: what it does, what it needs and gives, when it applies, and a command with what the program printed for it. `actuarial-science info <id>` shows the same inputs and outputs in the terminal.

Credibility (7)

Buhlmann credibility premium (known structural parameters) — `act_buhlmann_credibility`

The Buhlmann credibility factor Z = n/(n + k), k = EPV/VHM, and the credibility premium Z xbar + (1 - Z) mu from the overall mean, the expected process variance, the variance of the hypothetical means and n observations with mean xbar.

$$k = \frac{\mathrm{EPV}}{\mathrm{VHM}}, \quad Z = \frac{n}{n+k}, \quad P_C = Z\bar{X} + (1-Z)\mu$$

symbol		meaning	SI unit
`mu`	in	overall (collective) mean	—
`EPV`	in	expected process variance	—
`VHM`	in	variance of the hypothetical means	—
`n`	in	number of observations	—
`xbar`	in	mean of the observations	—
`k`	out	Buhlmann k = EPV/VHM	—
`Z`	out	credibility factor n/(n + k)	—
`premium`	out	Buhlmann credibility premium	—

Applies when: EPV, VHM > 0; n > 0.

```
$ actuarial-science solve act_buhlmann_credibility mu=1000 EPV=50000 VHM=10000 n=5 xbar=1200
k = 5
Z = 0.5
premium = 1100
  by: Buhlmann credibility premium (known structural parameters)  (k = EPV/VHM; Z = n/(n + k); P =
Z xbar + (1 - Z) mu)
  mu = 1000
  EPV = 50000
  VHM = 10000
  n = 5
  xbar = 1200
  assumes: EPV, VHM > 0; n > 0
```

In the worked example, mu = 1000, EPV = 50000, VHM = 10000, n = 5, xbar = 1200 go through the equation above and give k = 5, Z = 0.5, premium = 1100. It applies only when EPV, VHM > 0; n > 0: outside that it says what is wrong instead of returning a number.

Buhlmann-Straub credibility with exposures — `act_buhlmann_straub`

The nonparametric Buhlmann-Straub estimates of the expected process variance and of the variance of the hypothetical means from losses per unit of exposure and the exposures of r risks over n periods, each risk's credibility factor and its credibility premium (complement: the exposure-weighted overall mean).

$$\hat{v} = \frac{\sum_{i,j} m_{ij}(X_{ij} - \bar{X}_i)^2}{r(n-1)}, \quad \hat{a} = \frac{\sum_i m_i(\bar{X}_i - \bar{X})^2 - \hat{v}(r-1)}{m - \sum_i m_i^2/m}, \quad Z_i = \frac{m_i}{m_i + \hat{v}/\hat{a}}$$

symbol		meaning	SI unit
`X`	in	losses per unit of exposure: one row per risk, one column per period	—
`m`	in	exposures, the same shape as X	—
`EPV`	out	estimated expected process variance	—
`VHM`	out	estimated variance of the hypothetical means	—
`k`	out	k = EPV/VHM	—
`Z`	out	credibility factor of each risk m_i/(m_i + k) (a list)	—
`premiums`	out	credibility premium per unit of exposure of each risk (a list)	—
`mean`	out	exposure-weighted overall mean	—

Applies when: at least 2 risks and 2 periods; positive exposures; estimated VHM > 0.

```
$ actuarial-science solve act_buhlmann_straub "X=[[100, 120, 110], [200, 180, 230]]" "m=[[10, 20,
15], [30, 25, 20]]"
EPV = 7661.111111
VHM = 3834.197531
k = 1.998100267
Z = [0.957485509927,0.974050005645]
premiums = [114.590043128,200.466171022]
mean = 167.9166667
  by: Buhlmann-Straub credibility with exposures  (v = sum m_ij (X_ij - Xbar_i)^2 / (r(n-1)); a =
[sum m_i (Xbar_i - Xbar)^2 - v (r-1)] / (m - sum m_i^2/m); Z_i = m_i/(m_i + v/a))
  assumes: at least 2 risks and 2 periods; positive exposures; estimated VHM > 0
```

In the worked example, X = [100 120 110; 200 180 230], m = [10 20 15; 30 25 20] go through the equation above and give EPV = 7661.111111, VHM = 3834.197531, k = 1.998100267, Z = [0.9574855099, 0.9740500056]. It applies only when at least 2 risks and 2 periods; positive exposures; estimated VHM > 0: outside that it says what is wrong instead of returning a number.

Empirical Buhlmann credibility from a data table — `act_buhlmann_empirical`

The nonparametric estimates of the expected process variance and of the variance of the hypothetical means from r risks observed for n periods each, the Buhlmann k and Z, and each risk's credibility premium.

$$\hat{v} = \frac{1}{r(n-1)} \sum_{i,j} \left(X_{ij} - \bar{X}_i \right)^2, \quad \hat{a} = \frac{1}{r-1} \sum_{i} \left(\bar{X}_i - \bar{X} \right)^2 - \frac{\hat{v}}{n}$$

symbol		meaning	SI unit
`X`	in	observations: one row per risk, one column per period (rows of equal length)	—
`EPV`	out	estimated expected process variance	—
`VHM`	out	estimated variance of the hypothetical means	—
`k`	out	Buhlmann k = EPV/VHM	—
`Z`	out	credibility factor n/(n + k)	—
`premiums`	out	credibility premium of each risk (a list)	—
`mean`	out	overall mean	—

Applies when: at least 2 risks and 2 periods; estimated VHM > 0.

```
$ actuarial-science solve act_buhlmann_empirical "X=[[3, 5, 7], [6, 12, 9]]"
EPV = 6.5
VHM = 5.833333333
k = 1.114285714
Z = 0.7291666667
premiums = [5.54166666667,8.45833333333]
mean = 7
  by: Empirical Buhlmann credibility from a data table  (EPV = mean of the within-risk sample
variances; VHM = sample variance of the risk means - EPV/n; Z = n/(n + EPV/VHM))
  assumes: at least 2 risks and 2 periods; estimated VHM > 0
```

In the worked example, X = [3 5 7; 6 12 9] go through the equation above and give EPV = 6.5, VHM = 5.833333333, k = 1.114285714, Z = 0.7291666667. It applies only when at least 2 risks and 2 periods; estimated VHM > 0: outside that it says what is wrong instead of returning a number.

Limited fluctuation full credibility for severity and aggregate losses — `act_limited_fluctuation_severity_aggregate`

The expected number of claims needed for full credibility of the claim severity, of aggregate losses (pure premium) and of the claim frequency, for Poisson claim counts and a severity with the given mean and standard deviation.

$$n_S = n_0\left(1+\frac{\sigma_X^2}{\mu_X^2}\right), \quad n_0 = \left(\frac{z_{(1+p)/2}}{k}\right)^2$$

symbol		meaning	SI unit
`p`	in	probability level (90% or 0.9)	—
`k`	in	tolerance as a fraction of the mean (5% or 0.05)	—
`mean_x`	in	mean claim severity	—
`sd_x`	in	standard deviation of the claim severity	—
`claims_aggregate`	out	claims for full credibility of aggregate losses n0 (1 + CV^2)	—
`claims_severity`	out	claims for full credibility of severity n0 CV^2	—
`claims_frequency`	out	claims for full credibility of frequency n0	—

Applies when: 0 < p < 1; 0 < k < 1; Poisson claim counts.

```
$ actuarial-science solve act_limited_fluctuation_severity_aggregate p=0.9 k=0.05 mean_x=1000
sd_x=2000
claims_aggregate = 5411.086908
claims_severity = 4328.869527
claims_frequency = 1082.217382
  by: Limited fluctuation full credibility for severity and aggregate losses  (n0 = (z/k)^2;
aggregate: n0 (1 + (sd/mean)^2); severity: n0 (sd/mean)^2)
  p = 0.9
  k = 0.05
  mean_x = 1000
  sd_x = 2000
  assumes: 0 < p < 1; 0 < k < 1; Poisson claim counts
```

In the worked example, p = 0.9, k = 0.05, mean_x = 1000, sd_x = 2000 go through the equation above and give claims_aggregate = 5411.086908, claims_severity = 4328.869527, claims_frequency = 1082.217382. It applies only when 0 < p < 1; 0 < k < 1; Poisson claim counts: outside that it says what is wrong instead of returning a number.

Limited fluctuation full credibility standard for claim counts — `act_limited_fluctuation_full_credibility`

The expected number of claims needed for full credibility of the claim frequency so that the observed number is within a fraction k of its mean with probability p (Poisson claim counts).

$$n_F = \left(\frac{z_{(1+p)/2}}{k} \right)^2$$

symbol		meaning	SI unit
`p`	in	probability level (90% or 0.9)	—
`k`	in	tolerance as a fraction of the mean (5% or 0.05)	—
`n_full`	out	expected claims needed for full credibility (z/k)^2	—
`z`	out	standard normal quantile z_((1+p)/2)	—

Applies when: 0 < p < 1; 0 < k < 1; Poisson claim counts.

```
$ actuarial-science solve act_limited_fluctuation_full_credibility p=0.9 k=0.05
n_full = 1082.217382
z = 1.644853627
  by: Limited fluctuation full credibility standard for claim counts  (n_F = (z_((1+p)/2) / k)^2)
  p = 0.9
  k = 0.05
  assumes: 0 < p < 1; 0 < k < 1; Poisson claim counts
```

In the worked example, p = 0.9, k = 0.05 go through the equation above and give n_full = 1082.217382, z = 1.644853627. It applies only when 0 < p < 1; 0 < k < 1; Poisson claim counts: outside that it says what is wrong instead of returning a number.

Partial credibility (square root rule) — `act_partial_credibility`

The credibility factor Z = min(1, sqrt(n/n_F)) of an experience with n claims against a full credibility standard n_F, and the credibility-weighted estimate Z xbar + (1 - Z) M.

$$Z = \min\left(1, \sqrt{n/n_F}\right), \quad P_C = Z\bar{X} + (1 - Z)M$$

symbol		meaning	SI unit
`n`	in	observed (or expected) number of claims	—
`n_full`	in	full credibility standard in claims	—
`xbar`	in	observed experience (mean or rate)	—
`M`	in	complement of credibility (manual rate)	—
`Z`	out	credibility factor	—
`premium`	out	credibility-weighted estimate	—

Applies when: n >= 0; n_F > 0.

```
$ actuarial-science solve act_partial_credibility n=300 n_full=1082 xbar=800 M=1000
Z = 0.5265589476
premium = 894.6882105
  by: Partial credibility (square root rule)  (Z = min(1, sqrt(n / n_F)); estimate = Z xbar + (1 -
Z) M)
  n = 300
  n_full = 1082
  xbar = 800
  M = 1000
  assumes: n >= 0; n_F > 0
```

In the worked example, n = 300, n_full = 1082, xbar = 800, M = 1000 go through the equation above and give Z = 0.5265589476, premium = 894.6882105. It applies only when n >= 0; n_F > 0: outside that it says what is wrong instead of returning a number.

Poisson-gamma Bayesian credibility — `act_poisson_gamma_credibility`

The posterior (Bayesian) mean claim frequency of a risk whose Poisson claim rate has a gamma prior (alpha, theta: mean alpha theta), after observing claim counts in n periods, with the equivalent credibility factor and the posterior gamma parameters.

$$E[\Lambda \mid x] = \frac{(\alpha + \sum x_j)\,\theta}{1 + n\theta} = Z\bar{x} + (1 - Z)\alpha\theta, \quad Z = \frac{n\theta}{1 + n\theta}$$

symbol		meaning	SI unit
`alpha`	in	gamma prior shape alpha	—
`theta`	in	gamma prior scale theta	—
`counts`	in	observed claim counts, one per period (a list)	—
`post_mean`	out	posterior mean claim rate (alpha + sum x) theta/(1 + n theta)	—
`Z`	out	credibility factor n theta/(1 + n theta)	—

symbol		meaning	SI unit
`prior_mean`	out	prior mean alpha theta	—
`post_alpha`	out	posterior shape alpha + sum x	—
`post_theta`	out	posterior scale theta/(1 + n theta)	—

Applies when: alpha, theta > 0; whole-number counts.

```
$ actuarial-science solve act_poisson_gamma_credibility alpha=3 theta=0.5 "counts=[2, 1, 3]"
post_mean = 1.8
Z = 0.6
prior_mean = 1.5
post_alpha = 9
post_theta = 0.2
  by: Poisson-gamma Bayesian credibility  (posterior Gamma(alpha + sum x, theta/(1 + n theta)); its
mean = Z xbar + (1 - Z) alpha theta with Z = n theta/(1 + n theta))
  alpha = 3
  theta = 0.5
  assumes: alpha, theta > 0; whole-number counts
```

In the worked example, alpha = 3, theta = 0.5, counts = [2, 1, 3] go through the equation above and give post_mean = 1.8, Z = 0.6, prior_mean = 1.5, post_alpha = 9. It applies only when alpha, theta > 0; whole-number counts: outside that it says what is wrong instead of returning a number.

Interest theory (11)

Accumulated and present value of a single amount — `act_accumulated_value`

The value of an amount accumulated for t years at compound interest, the present value of the same amount due in t years, and the accumulation factor (1+i)^t.

$$AV = P(1+i)^t, \quad PV = P\,v^t$$

symbol		meaning	SI unit
`P`	in	amount	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`t`	in	time in years	—
`AV`	out	accumulated value P(1+i)^t	—
`PV`	out	present value P v^t	—
`factor`	out	accumulation factor (1+i)^t	—

Applies when: 0 < i < 1; 0 <= t <= 200.

```
$ actuarial-science solve act_accumulated_value P=1000 i=0.05 t=10
AV = 1628.894627
PV = 613.9132535
factor = 1.628894627
  by: Accumulated and present value of a single amount  (AV = P(1+i)^t; PV = P(1+i)^(-t))
  P = 1000
  i = 0.05
  t = 10
  assumes: 0 < i < 1; 0 <= t <= 200
```

In the worked example, P = 1000, i = 0.05, t = 10 go through the equation above and give AV = 1628.894627, PV = 613.9132535, factor = 1.628894627. It applies only when 0 < i < 1; 0 <= t <= 200: outside that it says what is wrong instead of returning a number.

Annuity-certain present and accumulated values — `act_annuity_certain`

The present value of n level payments of 1 at the end (immediate) or start (due) of each year, their accumulated values at time n, and the continuously payable annuity over n years.

$$a_{\overline{n}|} = \frac{1 - v^n}{i},\ \ddot{a}_{\overline{n}|} = \frac{1 - v^n}{d},\ s_{\overline{n}|} = \frac{(1+i)^n - 1}{i},\ \bar{a}_{\overline{n}|} = \frac{1 - v^n}{\delta}$$

symbol		meaning	SI unit
`n`	in	number of annual payments (term in years)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`a_n`	out	present value of an annuity-immediate a_n	—
`a_due`	out	present value of an annuity-due	—
`s_n`	out	accumulated value of an annuity-immediate s_n	—
`s_due`	out	accumulated value of an annuity-due	—
`a_cont`	out	present value of a continuous annuity over n years	—

Applies when: n >= 1; 0 < i < 1.

```
$ actuarial-science solve act_annuity_certain n=10 i=0.05
a_n = 7.721734929
a_due = 8.107821676
s_n = 12.57789254
s_due = 13.20678716
a_cont = 7.913208595
  by: Annuity-certain present and accumulated values  (a_n = (1 - v^n)/i, a-due = (1 - v^n)/d, s_n
= ((1+i)^n - 1)/i, s-due = ((1+i)^n - 1)/d, continuous = (1 - v^n)/delta)
  i = 0.05
  assumes: n >= 1; 0 < i < 1
```

In the worked example, n = 10, i = 0.05 go through the equation above and give a_n = 7.721734929, a_due = 8.107821676, s_n = 12.57789254, s_due = 13.20678716. It applies only when n >= 1; 0 < i < 1: outside that it says what is wrong instead of returning a number.

Effective annual rate from a nominal rate — `act_nominal_to_effective_rate`

The annual effective interest and discount rates, force of interest and discount factor equivalent to a nominal interest or discount rate convertible m times a year.

$$1+i=\left(1+\frac{i^{(m)}}{m}\right)^{m}=\left(1-\frac{d^{(m)}}{m}\right)^{-m}$$

symbol		meaning	SI unit
`rate`	in	nominal annual rate (6% or 0.06)	—
`m`	in	conversion periods per year (12 = monthly)	—
`kind`	in	the nominal rate is an interest rate i(m) or a discount rate d(m)	—
`i`	out	annual effective interest rate	—
`d`	out	annual effective discount rate	—
`delta`	out	force of interest	—
`v`	out	annual discount factor	—

Applies when: 0 < rate < 1; 1 <= m <= 365.

```
$ actuarial-science solve act_nominal_to_effective_rate rate=0.06 m=12 kind=interest
i = 0.06167781186
d = 0.05809466033
delta = 0.05985049813
v = 0.9419053397
  by: Effective annual rate from a nominal rate  (1 + i = (1 + i(m)/m)^m or (1 - d(m)/m)^(-m); d =
i/(1+i); delta = ln(1+i))
  rate = 0.06
  kind = interest
  assumes: 0 < rate < 1; 1 <= m <= 365
```

In the worked example, rate = 0.06, m = 12, kind = interest go through the equation above and give i = 0.06167781186, d = 0.05809466033, delta = 0.05985049813, v = 0.9419053397. It applies only when 0 < rate < 1; 1 <= m <= 365: outside that it says what is wrong instead of returning a number.

Effective rates from a constant force of interest — `act_force_of_interest`

The annual effective interest rate, discount rate and discount factor for a constant force of interest delta.

$$i=e^{\delta}-1,\quad d=1-e^{-\delta},\quad v=e^{-\delta}$$

symbol		meaning	SI unit
`delta`	in	constant force of interest per year (0.05 or 5%)	—
`i`	out	annual effective interest rate	—
`d`	out	annual effective discount rate	—
`v`	out	annual discount factor	—

Applies when: 0 < delta < 1.

```
$ actuarial-science solve act_force_of_interest delta=0.05
i = 0.05127109638
d = 0.0487705755
v = 0.9512294245
  by: Effective rates from a constant force of interest  (i = e^delta - 1, d = 1 - e^(-delta), v =
e^(-delta))
  delta = 0.05
  assumes: 0 < delta < 1
```

In the worked example, delta = 0.05 go through the equation above and give i = 0.05127109638, d = 0.0487705755, v = 0.9512294245. It applies only when 0 < delta < 1: outside that it says what is wrong instead of returning a number.

Equivalent rates from an effective annual interest rate — `act_interest_rate_conversion`

The effective discount rate, discount factor, force of interest and the nominal interest and discount rates convertible m times a year equivalent to an annual effective interest rate.

$$d = \frac{i}{1+i},\ \delta = \ln(1+i),\ i^{(m)} = m\left[(1+i)^{1/m} - 1\right],\ d^{(m)} = m\left[1 - v^{1/m}\right]$$

symbol		meaning	SI unit
`i`	in	annual effective interest rate (5% or 0.05)	—
`m`	in	conversion periods per year for the nominal rates (12 = monthly)	—
`d`	out	effective annual discount rate	—
`v`	out	annual discount factor 1/(1+i)	—
`delta`	out	force of interest ln(1+i)	—
`i_m`	out	nominal interest rate convertible m-thly	—
`d_m`	out	nominal discount rate convertible m-thly	—

Applies when: 0 < i < 1; 1 <= m <= 365.

```
$ actuarial-science solve act_interest_rate_conversion i=0.05 m=12
d = 0.04761904762
v = 0.9523809524
delta = 0.04879016417
i_m = 0.0488894854
d_m = 0.04869111179
  by: Equivalent rates from an effective annual interest rate  (d = i/(1+i), v = 1/(1+i), delta =
ln(1+i), i(m) = m[(1+i)^(1/m) - 1], d(m) = m[1 - v^(1/m)])
  i = 0.05
  assumes: 0 < i < 1; 1 <= m <= 365
```

In the worked example, i = 0.05, m = 12 go through the equation above and give d = 0.04761904762, v = 0.9523809524, delta = 0.04879016417, i_m = 0.0488894854. It applies only when 0 < i < 1; 1 <= m <= 365: outside that it says what is wrong instead of returning a number.

Increasing and decreasing annuities-certain — `act_increasing_decreasing_annuity`

The present values of an n-year annuity paying 1, 2, ..., n (increasing) and n, n-1, ..., 1 (decreasing), immediate and due.

$$(Ia)_{\overline{n}|} = \frac{\ddot{a}_{\overline{n}|} - nv^n}{i}, \quad (Da)_{\overline{n}|} = \frac{n - a_{\overline{n}|}}{i}$$

symbol		meaning	SI unit
`n`	in	term in years	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`Ia`	out	increasing annuity-immediate (Ia)_n	—
`Ia_due`	out	increasing annuity-due	—
`Da`	out	decreasing annuity-immediate (Da)_n	—
`Da_due`	out	decreasing annuity-due	—

Applies when: n >= 1; 0 < i < 1.

```
$ actuarial-science solve act_increasing_decreasing_annuity n=10 i=0.05
Ia = 39.3737828
Ia_due = 41.34247194
Da = 45.56530142
Da_due = 47.84356649
  by: Increasing and decreasing annuities-certain  ((Ia)_n = (a-due_n - n v^n)/i; (Da)_n = (n -
a_n)/i; due values times (1+i))
  i = 0.05
  assumes: n >= 1; 0 < i < 1
```

In the worked example, n = 10, i = 0.05 go through the equation above and give Ia = 39.3737828, Ia_due = 41.34247194, Da = 45.56530142, Da_due = 47.84356649. It applies only when n >= 1; 0 < i < 1: outside that it says what is wrong instead of returning a number.

Loan amortization: level payment, interest and principal, outstanding balance — `act_loan_amortization`

The level annual payment that repays a loan over n years, the interest and principal parts of payment t and the outstanding balance just after it.

$$R = \frac{L}{a_{\overline{n}|}}, \quad I_t = R\left(1 - v^{n-t+1}\right), \quad P_t = R\,v^{n-t+1}, \quad B_t = R\,a_{\overline{n-t}|}$$

symbol		meaning	SI unit
`L`	in	loan amount	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`n`	in	number of annual payments	—
`t`	in	payment number of interest (1 to n)	—
`R`	out	level annual payment	—
`interest_t`	out	interest part of payment t	—
`principal_t`	out	principal repaid in payment t	—

symbol		meaning	SI unit
`balance_t`	out	outstanding balance just after payment t	—
`total_interest`	out	total interest paid over the loan	—

Applies when: L > 0; 0 < i < 1; 1 <= t <= n.

```
$ actuarial-science solve act_loan_amortization L=100000 i=0.06 n=20 t=5
R = 8718.455698
interest_t = 5286.468013
principal_t = 3431.987685
balance_t = 84675.81252
total_interest = 74369.11395
  by: Loan amortization: level payment, interest and principal, outstanding balance  (R = L/a_n;
balance before payment t = R a_(n-t+1); interest = i x balance; principal = R v^(n-t+1); balance
after = R a_(n-t))
  L = 100000
  i = 0.06
  assumes: L > 0; 0 < i < 1; 1 <= t <= n
```

In the worked example, L = 100000, i = 0.06, n = 20, t = 5 go through the equation above and give R = 8718.455698, interest_t = 5286.468013, principal_t = 3431.987685, balance_t = 84675.81252. It applies only when L > 0; 0 < i < 1; 1 <= t <= n: outside that it says what is wrong instead of returning a number.

Macaulay and modified duration and convexity of cash flows — `act_duration_convexity`

The present value, Macaulay duration, modified duration and convexity (second derivative of price with respect to i over price) of a set of non-negative cash flows.

$$D_{mac} = \frac{\sum t\, C_t v^t}{P}, \quad D_{mod} = \frac{D_{mac}}{1+i}, \quad C = \frac{\sum t(t+1) C_t v^{t+2}}{P}$$

symbol		meaning	SI unit
`amounts`	in	cash flow amounts (a list, non-negative)	—
`times`	in	payment times in years (a list)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`PV`	out	present value	—
`macaulay`	out	Macaulay duration (years)	—
`modified`	out	modified duration -P'(i)/P	—
`convexity`	out	convexity P''(i)/P	—

Applies when: amounts >= 0 with a positive total; one time per amount; 0 < i < 1.

```
$ actuarial-science solve act_duration_convexity "amounts=[5, 5, 105]" "times=[1, 2, 3]" i=0.05
PV = 100
macaulay = 2.859410431
modified = 2.723248029
convexity = 10.2056242
  by: Macaulay and modified duration and convexity of cash flows  (D_mac = sum t C v^t / P; D_mod =
D_mac/(1+i); convexity = sum t(t+1) C v^(t+2) / P)
  i = 0.05
  assumes: amounts >= 0 with a positive total; one time per amount; 0 < i < 1
```

In the worked example, amounts = [5, 5, 105], times = [1, 2, 3], i = 0.05 go through the equation above and give PV = 100, macaulay = 2.859410431, modified = 2.723248029, convexity = 10.2056242. It applies only when amounts >= 0 with a positive total; one time per amount; 0 < i < 1: outside that it says what is wrong instead of returning a number.

Present value of a perpetuity — `act_perpetuity`

The present value of payments of 1 a year forever, at the end (immediate) or start (due) of each year, and paid continuously.

$$a_{\overline{\infty}|} = \frac{1}{i}, \quad \ddot{a}_{\overline{\infty}|} = \frac{1}{d}, \quad \bar{a}_{\overline{\infty}|} = \frac{1}{\delta}$$

symbol		meaning	SI unit
`i`	in	annual effective interest rate (5% or 0.05)	—
`a_inf`	out	perpetuity-immediate 1/i	—
`a_due_inf`	out	perpetuity-due 1/d	—
`a_cont_inf`	out	continuous perpetuity 1/delta	—

Applies when: 0 < i < 1.

```
$ actuarial-science solve act_perpetuity i=0.05
a_inf = 20
a_due_inf = 21
a_cont_inf = 20.49593431
  by: Present value of a perpetuity  (a_inf = 1/i, a-due_inf = 1/d = (1+i)/i, continuous 1/delta)
  i = 0.05
  assumes: 0 < i < 1
```

In the worked example, i = 0.05 go through the equation above and give a_inf = 20, a_due_inf = 21, a_cont_inf = 20.49593431. It applies only when 0 < i < 1: outside that it says what is wrong instead of returning a number.

Present value of a series of cash flows — `act_present_value_cash_flows`

The present value at time 0 of amounts paid at stated times (years), and their accumulated value at the last payment time.

$$PV = \sum_k C_k v^{t_k}$$

symbol		meaning	SI unit
`amounts`	in	cash flow amounts (a list)	—
`times`	in	payment times in years (a list)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`PV`	out	present value at time 0	—
`AV_at_last`	out	accumulated value at the last payment time	—

Applies when: one time per amount; 0 <= t <= 200; 0 < i < 1.

```
$ actuarial-science solve act_present_value_cash_flows "amounts=[100, 200, 300]" "times=[1, 2, 3]"
i=0.05
PV = 535.7952705
AV_at_last = 620.25
  by: Present value of a series of cash flows  (PV = sum C_k v^(t_k); AV = PV (1+i)^(t_max))
  i = 0.05
  assumes: one time per amount; 0 <= t <= 200; 0 < i < 1
```

In the worked example, amounts = [100, 200, 300], times = [1, 2, 3], i = 0.05 go through the equation above and give PV = 535.7952705, AV_at_last = 620.25. It applies only when one time per amount; 0 <= t <= 200; 0 < i < 1: outside that it says what is wrong instead of returning a number.

Yield rate of a level annuity — `act_annuity_yield`

The annual effective interest rate at which n level payments R at the ends of the years are worth a given present value.

$$R\,\frac{1-(1+i)^{-n}}{i} = PV$$

symbol		meaning	SI unit
`PV`	in	present value (price) of the annuity	—
`R`	in	level annual payment	—
`n`	in	number of annual payments	—
`i`	out	yield rate (annual effective)	—
`a_n`	out	annuity factor PV/R	—

Applies when: 0 < PV < n R; yield below 99%.

```
$ actuarial-science solve act_annuity_yield PV=7721.73 R=1000 n=10
i = 0.05000013145
a_n = 7.72173
  by: Yield rate of a level annuity  (solve R a_n(i) = PV for i by bisection (a_n decreases in i,
so the root is unique))
  PV = 7721.73
  R = 1000
  assumes: 0 < PV < n R; yield below 99%
```

In the worked example, PV = 7721.73, R = 1000, n = 10 go through the equation above and give i = 0.05000013145, a_n = 7.72173. It applies only when 0 < PV < n R; yield below 99%: outside that it says what is wrong instead of returning a number.

Life annuities (5)

Deferred whole life annuity-due — `act_deferred_annuity`

The actuarial present value of a whole life annuity-due of P a year starting u years from now if the life is then alive (u|ä_x = uE_x ä_(x+u)).

$$ {}_{u|}\ddot{a}_x = \sum_{k \geq u} v^k \, {}_kp_x = {}_uE_x \, \ddot{a}_{x+u} $$

symbol		meaning	SI unit
`q`	in	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`u`	in	deferment in years	—
`P`	in	annual payment	—
`ua_due`	out	deferred annuity-due u|ä_x	—
`EPV`	out	actuarial present value P u|ä_x	—

Applies when: the table closes; u < length of the table; 0 < i < 1.

```
$ actuarial-science solve act_deferred_annuity "q=[0.1, 0.2, 0.5, 1]" i=0.05 u=2 P=1000
ua_due = 0.96404276
EPV = 964.04276
  by: Deferred whole life annuity-due  (u|ä_x = sum_(k>=u) v^k k_p_x = u_E_x ä_(x+u))
  i = 0.05
  P = 1000
  assumes: the table closes; u < length of the table; 0 < i < 1
```

In the worked example, q = [0.1, 0.2, 0.5, 1], i = 0.05, u = 2, P = 1000 go through the equation above and give ua_due = 0.96404276, EPV = 964.04276. It applies only when the table closes; u < length of the table; 0 < i < 1: outside that it says what is wrong instead of returning a number.

m-thly whole life annuity-due under UDD — `act_mthly_annuity_udd`

The actuarial present value of a whole life annuity-due paying 1/m at the start of every 1/m of a year, from an annual life table under the uniform distribution of deaths (ä^(m)_x = alpha(m) ä_x - beta(m)).

$$ \ddot{a}_x^{(m)} = \alpha(m)\,\ddot{a}_x - \beta(m), \quad \alpha(m) = \frac{id}{i^{(m)}d^{(m)}},\ \beta(m) = \frac{i - i^{(m)}}{i^{(m)}d^{(m)}} $$

symbol		meaning	SI unit
`q`	in	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	—
`i`	in	annual effective interest rate (5% or 0.05)	—

symbol		meaning	SI unit
`m`	in	payments per year (12 = monthly)	—
`a_m`	out	m-thly annuity-due ä^(m)_x (total 1 a year)	—
`alpha`	out	alpha(m) = i d/(i(m) d(m))	—
`beta`	out	beta(m) = (i - i(m))/(i(m) d(m))	—
`a_due`	out	annual annuity-due ä_x	—

Applies when: the table closes; 0 < i < 1; 1 <= m <= 365; UDD within each year.

```
$ actuarial-science solve act_mthly_annuity_udd "q=[0.1, 0.2, 0.5, 1]" i=0.05 m=12
a_m = 2.355233403
alpha = 1.000197011
beta = 0.4665080196
a_due = 2.821185617
  by: m-thly whole life annuity-due under UDD  (ä^(m)_x = alpha(m) ä_x - beta(m))
  i = 0.05
  assumes: the table closes; 0 < i < 1; 1 <= m <= 365; UDD within each year
```

In the worked example, q = [0.1, 0.2, 0.5, 1], i = 0.05, m = 12 go through the equation above and give a_m = 2.355233403, alpha = 1.000197011, beta = 0.4665080196, a_due = 2.821185617. It applies only when the table closes; 0 < i < 1; 1 <= m <= 365; UDD within each year: outside that it says what is wrong instead of returning a number.

Temporary life annuity — `act_temporary_annuity`

The actuarial present values of an n-year temporary life annuity-due (ä_(x:n)) and annuity-immediate (a_(x:n)), and the value of an annual payment P paid in advance.

$$\ddot{a}_{x:\overline{n}|} = \sum_{k=0}^{n-1} v^k \, {}_kp_x, \quad a_{x:\overline{n}|} = \ddot{a}_{x:\overline{n}|} - 1 + {}_nE_x$$

symbol		meaning	SI unit
`q`	in	death probabilities q_x, q_(x+1), ... for at least n years (a list)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`n`	in	number of years	—
`P`	in	annual payment	—
`a_due`	out	temporary annuity-due ä_(x:n)	—
`a_imm`	out	temporary annuity-immediate a_(x:n)	—
`EPV`	out	actuarial present value P ä_(x:n)	—

Applies when: at least n death probabilities; 0 < i < 1; P > 0.

```
$ actuarial-science solve act_temporary_annuity "q=[0.1, 0.2, 0.5, 1]" i=0.05 n=3 P=1000
a_due = 2.510204082
a_imm = 1.821185617
EPV = 2510.204082
  by: Temporary life annuity  (ä_(x:n) = sum_(k<n) v^k k_p_x; a_(x:n) = ä_(x:n) - 1 + n_E_x)
  i = 0.05
  P = 1000
  assumes: at least n death probabilities; 0 < i < 1; P > 0
```

In the worked example, q = [0.1, 0.2, 0.5, 1], i = 0.05, n = 3, P = 1000 go through the equation above and give a_due = 2.510204082, a_imm = 1.821185617, EPV = 2510.204082. It applies only when at least n death probabilities; 0 < i < 1; P > 0: outside that it says what is wrong instead of returning a number.

Whole life annuity — `act_whole_life_annuity`

The actuarial present values of a whole life annuity-due (ä_x) and annuity-immediate (a_x) for a life aged x, the value of an annual payment P, and the standard deviation of the present value of the annuity-due payments.

$$\ddot{a}_x = \sum_{k\geq 0} v^k \,{}_kp_x, \quad a_x = \ddot{a}_x - 1, \quad \mathrm{Var}(Y) = \frac{{}^2A_x - A_x^2}{d^2}$$

symbol		meaning	SI unit
`q`	in	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`P`	in	annual payment	—
`a_due`	out	whole life annuity-due ä_x	—
`a_imm`	out	whole life annuity-immediate a_x	—
`EPV`	out	actuarial present value P ä_x	—
`sd`	out	standard deviation of the annuity-due present value	—

Applies when: the table closes; 0 < i < 1; P > 0.

```
$ actuarial-science solve act_whole_life_annuity "q=[0.1, 0.2, 0.5, 1]" i=0.05 P=1000
a_due = 2.821185617
a_imm = 1.821185617
EPV = 2821.185617
sd = 872.3551839
  by: Whole life annuity  (ä_x = sum_k v^k k_p_x; a_x = ä_x - 1; Var = P^2 (2A_x - A_x^2)/d^2)
  i = 0.05
  P = 1000
  assumes: the table closes; 0 < i < 1; P > 0
```

In the worked example, q = [0.1, 0.2, 0.5, 1], i = 0.05, P = 1000 go through the equation above and give a_due = 2.821185617, a_imm = 1.821185617, EPV = 2821.185617, sd = 872.3551839. It applies only when the table closes; 0 < i < 1; P > 0: outside that it says what is wrong instead of returning a number.

Woolhouse approximation for an m-thly annuity — `act_woolhouse_approximation`

Woolhouse's two- and three-term approximations of the m-thly whole life annuity-due ä^(m)_x from the annual annuity-due ä_x, the interest rate and the force of mortality at x.

$$\ddot{a}_x^{(m)} \approx \ddot{a}_x - \frac{m-1}{2m} - \frac{m^2-1}{12m^2}(\delta + \mu_x)$$

symbol		meaning	SI unit
`a_due`	in	annual whole life annuity-due ä_x	—
`m`	in	payments per year (12 = monthly)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`mu`	in	force of mortality at age x (per year)	—
`a_m3`	out	three-term Woolhouse approximation	—
`a_m2`	out	two-term approximation ä_x - (m-1)/(2m)	—

Applies when: ä_x >= 1; 1 <= m <= 365; 0 < i < 1; 0 <= mu < 5.

```
$ actuarial-science solve act_woolhouse_approximation a_due=14.8166 m=12 i=0.05 mu=0.00473
a_m3 = 14.35383763
a_m2 = 14.35826667
  by: Woolhouse approximation for an m-thly annuity  (ä^(m)_x ≈ ä_x - (m-1)/(2m) - (m^2-1)/(12 m^2)
(delta + mu_x))
  a_due = 14.8166
  i = 0.05
  mu = 0.00473
  assumes: ä_x >= 1; 1 <= m <= 365; 0 < i < 1; 0 <= mu < 5
```

In the worked example, a_due = 14.8166, m = 12, i = 0.05, mu = 0.00473 go through the equation above and give a_m3 = 14.35383763, a_m2 = 14.35826667. It applies only when ä_x >= 1; 1 <= m <= 365; 0 < i < 1; 0 <= mu < 5: outside that it says what is wrong instead of returning a number.

Life insurance (5)

Continuous whole life insurance and annuity under UDD — `act_continuous_insurance_udd`

The actuarial present value of a whole life insurance paid at the moment of death (Ā_x = (i/delta) A_x) and of a continuous whole life annuity, from an annual life table with deaths uniformly distributed within each year.

$$\bar{A}_x = \frac{i}{\delta} A_x, \qquad \bar{a}_x = \frac{1 - \bar{A}_x}{\delta}$$

symbol		meaning	SI unit
`q`	in	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`A_bar`	out	insurance payable at the moment of death Ā_x	—
`A`	out	insurance payable at the end of the year of death A_x	—
`a_bar`	out	continuous whole life annuity ā_x	—

Applies when: the table closes; 0 < i < 1; uniform distribution of deaths within each year.

```
$ actuarial-science solve act_continuous_insurance_udd "q=[0.1, 0.2, 0.5, 1]" i=0.05
A_bar = 0.8871232988
A = 0.8656578278
a_bar = 2.313513453
  by: Continuous whole life insurance and annuity under UDD  (Ā_x = (i/delta) A_x; ā_x = (1 -
Ā_x)/delta)
  i = 0.05
  assumes: the table closes; 0 < i < 1; uniform distribution of deaths within each year
```

In the worked example, q = [0.1, 0.2, 0.5, 1], i = 0.05 go through the equation above and give A_bar = 0.8871232988, A = 0.8656578278, a_bar = 2.313513453. It applies only when the table closes; 0 < i < 1; uniform distribution of deaths within each year: outside that it says what is wrong instead of returning a number.

Deferred whole life insurance — `act_deferred_insurance`

The actuarial present value of a whole life insurance of S that pays only if death occurs after u years (u|A_x = uE_x A_(x+u)).

$$ {}_{u|}A_x = \sum_{k \geq u} v^{k+1}\, {}_kp_x\, q_{x+k} = {}_uE_x\, A_{x+u} $$

symbol		meaning	SI unit
`q`	in	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`u`	in	deferment in years	—
`S`	in	sum insured	—
`uA`	out	deferred insurance u|A_x per unit	—
`EPV`	out	actuarial present value S u|A_x	—
`uEx`	out	pure endowment u_E_x	—

Applies when: the table closes; u < length of the table; 0 < i < 1.

```
$ actuarial-science solve act_deferred_insurance "q=[0.1, 0.2, 0.5, 1]" i=0.05 u=1 S=1000
uA = 0.7704197325
EPV = 770.4197325
uEx = 0.8571428571
  by: Deferred whole life insurance  (u|A_x = sum_(k>=u) v^(k+1) k_p_x q_(x+k) = u_E_x A_(x+u))
  i = 0.05
  S = 1000
  assumes: the table closes; u < length of the table; 0 < i < 1
```

In the worked example, q = [0.1, 0.2, 0.5, 1], i = 0.05, u = 1, S = 1000 go through the equation above and give uA = 0.7704197325, EPV = 770.4197325, uEx = 0.8571428571. It applies only when the table closes; u < length of the table; 0 < i < 1: outside that it says what is wrong instead of returning a number.

Endowment insurance (fully discrete) — `act_endowment_insurance`

The actuarial present value of an n-year endowment insurance paying S at the end of the year of death within n years or at time n on survival, split into its term insurance and pure endowment parts, with the second moment and standard deviation.

$$A_{x:\overline{n}|} = A^{1}_{x:\overline{n}|} + {}_{n}E_{x}$$

symbol		meaning	SI unit
`q`	in	death probabilities q_x, q_(x+1), ... for at least n years (a list)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`n`	in	term in years	—
`S`	in	sum insured	—
`A`	out	endowment insurance A_(x:n) per unit	—
`A1`	out	term insurance part A^1_(x:n)	—
`nEx`	out	pure endowment part n_E_x	—
`A2`	out	second moment at double the force of interest	—
`EPV`	out	actuarial present value S A_(x:n)	—
`sd`	out	standard deviation of the present value	—

Applies when: at least n death probabilities; 0 < i < 1; S > 0.

```
$ actuarial-science solve act_endowment_insurance "q=[0.1, 0.2, 0.5, 1]" i=0.05 n=2 S=1000
A = 0.9115646258
A1 = 0.2585034014
nEx = 0.6530612245
A2 = 0.8311351752
EPV = 911.5646258
sd = 13.60544218
  by: Endowment insurance (fully discrete)  (A_(x:n) = A^1_(x:n) + n_E_x)
  i = 0.05
  S = 1000
  assumes: at least n death probabilities; 0 < i < 1; S > 0
```

In the worked example, q = [0.1, 0.2, 0.5, 1], i = 0.05, n = 2, S = 1000 go through the equation above and give A = 0.9115646258, A1 = 0.2585034014, nEx = 0.6530612245, A2 = 0.8311351752. It applies only when at least n death probabilities; 0 < i < 1; S > 0: outside that it says what is wrong instead of returning a number.

Term insurance (fully discrete) — `act_term_insurance`

The actuarial present value of an n-year term insurance paying S at the end of the year of death if death occurs within n years, its second moment and the standard deviation of the present value.

$$A^{1}_{x:\overline{n}|} = \sum_{k=0}^{n-1} v^{k+1}\ {}_{k}p_{x}\ q_{x+k}$$

symbol		meaning	SI unit
`q`	in	death probabilities q_x, q_(x+1), ... for at least n years (a list)	—

symbol		meaning	SI unit
`i`	in	annual effective interest rate (5% or 0.05)	—
`n`	in	term in years	—
`S`	in	sum insured	—
`A1`	out	term insurance A^1_(x:n) per unit	—
`A1_2`	out	second moment at double the force of interest	—
`EPV`	out	actuarial present value S A^1_(x:n)	—
`sd`	out	standard deviation of the present value	—

Applies when: at least n death probabilities; 0 < i < 1; S > 0.

```
$ actuarial-science solve act_term_insurance "q=[0.1, 0.2, 0.5, 1]" i=0.05 n=2 S=1000
A1 = 0.2585034014
A1_2 = 0.2387893933
EPV = 258.5034014
sd = 414.6870926
  by: Term insurance (fully discrete)  (A^1_(x:n) = sum_(k<n) v^(k+1) k_p_x q_(x+k))
  i = 0.05
  S = 1000
  assumes: at least n death probabilities; 0 < i < 1; S > 0
```

In the worked example, q = [0.1, 0.2, 0.5, 1], i = 0.05, n = 2, S = 1000 go through the equation above and give A1 = 0.2585034014, A1_2 = 0.2387893933, EPV = 258.5034014, sd = 414.6870926. It applies only when at least n death probabilities; 0 < i < 1; S > 0: outside that it says what is wrong instead of returning a number.

Whole life insurance (fully discrete) — `act_whole_life_insurance`

The actuarial present value of a whole life insurance paying S at the end of the year of death of a life aged x (A_x), its second moment at double the force of interest, and the standard deviation of the present value of the benefit.

$$A_x = \sum_{k\geq 0} v^{k+1}\,{}_kp_x\,q_{x+k}, \quad \operatorname{Var}(Z) = S^2\left({}^2A_x - A_x^2\right)$$

symbol		meaning	SI unit
`q`	in	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`S`	in	sum insured	—
`A`	out	A_x per unit sum insured	—
`A2`	out	second moment 2A_x (at double the force of interest)	—
`EPV`	out	actuarial present value S A_x	—
`sd`	out	standard deviation of the present value S sqrt(2A - A^2)	—

Applies when: the table closes (last q = 1); 0 < i < 1; S > 0.

```
$ actuarial-science solve act_whole_life_insurance "q=[0.1, 0.2, 0.5, 1]" i=0.05 S=1000
A = 0.8656578278
A2 = 0.7510891064
EPV = 865.6578278
sd = 41.54072304
  by: Whole life insurance (fully discrete)  (A_x = sum_k v^(k+1) k_p_x q_(x+k); 2A_x with v^2; Var
Z = S^2 (2A - A^2))
  i = 0.05
  S = 1000
  assumes: the table closes (last q = 1); 0 < i < 1; S > 0
```

In the worked example, q = [0.1, 0.2, 0.5, 1], i = 0.05, S = 1000 go through the equation above and give A = 0.8656578278, A2 = 0.7510891064, EPV = 865.6578278, sd = 41.54072304. It applies only when the table closes (last q = 1); 0 < i < 1; S > 0: outside that it says what is wrong instead of returning a number.

Life tables (9)

Commutation functions D, N, C and M — `act_commutation_functions`

The commutation functions D_x = v^x l_x, N_x, C_x = v^(x+1) d_x and M_x at age x of a complete life table, with the whole life insurance A_x = M_x/D_x and annuity-due ä_x = N_x/D_x.

$$D_x = v^x l_x,\ C_x = v^{x+1} d_x,\ N_x = \sum_{y \geq x} D_y,\ M_x = \sum_{y \geq x} C_y,\ A_x = \frac{M_x}{D_x},\ \ddot{a}_x = \frac{N_x}{D_x}$$

symbol		meaning	SI unit
`lx`	in	survivors at consecutive integer ages from x0, ending with 0 (a list)	—
`x0`	in	age of the first entry of lx	—
`x`	in	age at which the functions are wanted	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`D_x`	out	D_x = v^x l_x	—
`N_x`	out	N_x = sum of D_y for y >= x	—
`C_x`	out	C_x = v^(x+1) d_x	—
`M_x`	out	M_x = sum of C_y for y >= x	—
`A_x`	out	whole life insurance M_x/D_x	—
`a_due_x`	out	whole life annuity-due N_x/D_x	—

Applies when: the table closes (last lx = 0); x inside the table; 0 < i < 1.

```
$ actuarial-science solve act_commutation_functions "lx=[1000, 900, 700, 400, 0]" x0=95 x=95 i=0.05
D_x = 9.705470376
N_x = 27.54022849
C_x = 0.924330512
M_x = 8.394030925
A_x = 0.8648762604
a_due_x = 2.837598531
  by: Commutation functions D, N, C and M  (D_y = v^y l_y, C_y = v^(y+1) d_y, N_x = sum D_y, M_x =
sum C_y; A_x = M_x/D_x, ä_x = N_x/D_x)
  i = 0.05
  assumes: the table closes (last lx = 0); x inside the table; 0 < i < 1
```

In the worked example, lx = [1000, 900, 700, 400, 0], x0 = 95, x = 95, i = 0.05 go through the equation above and give D_x = 9.705470376, N_x = 27.54022849, C_x = 0.924330512, M_x = 8.394030925. It applies only when the table closes (last lx = 0); x inside the table; 0 < i < 1: outside that it says what is wrong instead of returning a number.

Curtate and complete expectation of life — `act_life_expectancy`

The curtate expectation of life e_x (whole years lived) and the complete expectation e°_x under a uniform distribution of deaths within each year, for a life aged x in a complete life table.

$$e_x = \sum_{k \geq 1} {}_kp_x, \quad \mathring{e}_x = e_x + \tfrac{1}{2}$$

symbol		meaning	SI unit
`lx`	in	survivors at consecutive integer ages from x0, ending with 0 (a list)	—
`x0`	in	age of the first entry of lx	—
`x`	in	age of the life	—
`e_curtate`	out	curtate expectation of life e_x (years)	—
`e_complete`	out	complete expectation of life under UDD (years)	—

Applies when: the table closes (last lx = 0); x inside the table.

```
$ actuarial-science solve act_life_expectancy "lx=[1000, 900, 700, 400, 0]" x0=95 x=95
e_curtate = 2
e_complete = 2.5
  by: Curtate and complete expectation of life  (e_x = sum_(k>=1) l_(x+k)/l_x; e°_x = e_x + 1/2
under UDD)
  assumes: the table closes (last lx = 0); x inside the table
```

In the worked example, lx = [1000, 900, 700, 400, 0], x0 = 95, x = 95 go through the equation above and give e_curtate = 2, e_complete = 2.5. It applies only when the table closes (last lx = 0); x inside the table: outside that it says what is wrong instead of returning a number.

Death probability from the central death rate — `act_central_death_rate`

The one-year death probability q_x corresponding to a central death rate m_x (deaths per year lived), under a uniform distribution of deaths or a constant force of mortality within the year.

$$\text{UDD: } q_x = \frac{2m_x}{2 + m_x}, \qquad \text{constant force: } q_x = 1 - e^{-m_x}$$

symbol		meaning	SI unit
`m`	in	central death rate m_x (deaths per person-year)	—
`assumption`	in	fractional-age assumption: udd or constant_force	—
`q`	out	one-year death probability q_x	—
`p`	out	one-year survival probability p_x	—

Applies when: m > 0; m < 2 under UDD.

```
$ actuarial-science solve act_central_death_rate m=0.02 assumption=udd
q = 0.0198019802
p = 0.9801980198
  by: Death probability from the central death rate  (UDD: q = 2m/(2+m); constant force: q = 1 -
e^(-m))
  m = 0.02
  assumption = udd
  assumes: m > 0; m < 2 under UDD
```

In the worked example, m = 0.02, assumption = udd go through the equation above and give q = 0.0198019802, p = 0.9801980198. It applies only when m > 0; m < 2 under UDD: outside that it says what is wrong instead of returning a number.

Deferred death probability u|t q x — `act_deferred_death_probability`

The probability that a life aged x survives u years and then dies within the following t years, from survivor numbers lx starting at age x0.

$${}_{u|t}q_x = \frac{l_{x+u} - l_{x+u+t}}{l_x} = {}_{u}p_x \; {}_{t}q_{x+u}$$

symbol		meaning	SI unit
`lx`	in	survivors at consecutive integer ages from x0 (a list)	—
`x0`	in	age of the first entry of lx	—
`x`	in	age of the life	—
`u`	in	years of deferment	—
`t`	in	length of the death interval in years	—
`prob`	out	deferred death probability u|t q_x	—
`upx`	out	probability of surviving the deferment	—

Applies when: x + u + t inside the table; t >= 1.

```
$ actuarial-science solve act_deferred_death_probability "lx=[1000, 950, 880, 780, 650]" x0=60 x=60
u=2 t=2
prob = 0.23
upx = 0.88
  by: Deferred death probability u|t q x  (u|t q_x = (l_(x+u) - l_(x+u+t))/l_x = u_p_x * t_q_(x+u))
  assumes: x + u + t inside the table; t >= 1
```

In the worked example, lx = [1000, 950, 880, 780, 650], x0 = 60, x = 60, u = 2, t = 2 go through the equation above and give prob = 0.23, upx = 0.88. It applies only when x + u + t inside the table; t >= 1: outside that it says what is wrong instead of returning a number.

Life table functions dx, qx and px from survivor numbers — `act_life_table_functions`

The numbers dying, the one-year death probabilities and the one-year survival probabilities at each age of a life table given by its survivor numbers lx.

$$d_x = l_x - l_{x+1}, \quad q_x = \frac{d_x}{l_x}, \quad p_x = \frac{l_{x+1}}{l_x}$$

symbol		meaning	SI unit
`lx`	in	survivors at consecutive integer ages (a list)	—
`dx`	out	deaths between ages x and x+1 (a list)	—
`qx`	out	one-year death probabilities (a list)	—
`px`	out	one-year survival probabilities (a list)	—

Applies when: lx non-increasing; only the last lx may be 0.

```
$ actuarial-science solve act_life_table_functions "lx=[1000, 950, 880, 780]"
dx = [50.0,70.0,100.0]
qx = [0.05,0.0736842105263,0.113636363636]
px = [0.95,0.926315789474,0.886363636364]
  by: Life table functions dx, qx and px from survivor numbers  (d_x = l_x - l_(x+1), q_x =
d_x/l_x, p_x = l_(x+1)/l_x)
  assumes: lx non-increasing; only the last lx may be 0
```

In the worked example, lx = [1000, 950, 880, 780] go through the equation above and give dx = [50, 70, 100], qx = [0.05, 0.07368421053, 0.1136363636], px = [0.95, 0.9263157895, 0.8863636364]. It applies only when lx non-increasing; only the last lx may be 0: outside that it says what is wrong instead of returning a number.

Probability of surviving t years from a life table — `act_survival_probability`

The probability that a life aged x survives t more years, the probability of dying within t years, and the number of deaths between ages x and x+t, from survivor numbers lx starting at age x0.

$${}_tp_x = \frac{l_{x+t}}{l_x}, \quad {}_tq_x = \frac{l_x - l_{x+t}}{l_x}$$

symbol		meaning	SI unit
`lx`	in	survivors at consecutive integer ages from x0 (a list)	—
`x0`	in	age of the first entry of lx	—
`x`	in	age of the life	—
`t`	in	number of years	—
`tpx`	out	probability of surviving t years	—
`tqx`	out	probability of dying within t years	—
`deaths`	out	number dying between ages x and x+t	—

Applies when: x0 <= x and x + t inside the table; l_x > 0.

```
$ actuarial-science solve act_survival_probability "lx=[1000, 950, 880, 780]" x0=60 x=61 t=2
tpx = 0.8210526316
tqx = 0.1789473684
deaths = 170
  by: Probability of surviving t years from a life table  (t_p_x = l_(x+t)/l_x; t_q_x = 1 - t_p_x)
  assumes: x0 <= x and x + t inside the table; l_x > 0
```

In the worked example, lx = [1000, 950, 880, 780], x0 = 60, x = 61, t = 2 go through the equation above and give tpx = 0.8210526316, tqx = 0.1789473684, deaths = 170. It applies only when x0 <= x and x + t inside the table; l_x > 0: outside that it says what is wrong instead of returning a number.

Pure endowment (actuarial discount factor) — `act_pure_endowment`

The actuarial present value of 1 paid at time n if a life aged x is then alive, nEx = v^n n_p_x, and the actuarial accumulation factor 1/nEx.

$$ {}_nE_x = v^n \, {}_np_x = v^n \frac{l_{x+n}}{l_x} $$

symbol		meaning	SI unit
`lx`	in	survivors at consecutive integer ages from x0 (a list)	—
`x0`	in	age of the first entry of lx	—
`x`	in	age of the life	—
`n`	in	years until the payment	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`nEx`	out	pure endowment n_E_x = v^n n_p_x	—
`accumulation`	out	actuarial accumulation factor 1/n_E_x	—

Applies when: x + n inside the table; l_(x+n) > 0; 0 < i < 1.

```
$ actuarial-science solve act_pure_endowment "lx=[100000, 99500, 98900, 98200, 97300]" x0=40 x=40
n=4 i=0.05
nEx = 0.800489508
accumulation = 1.249235612
  by: Pure endowment (actuarial discount factor)  (nEx = v^n l_(x+n)/l_x)
  i = 0.05
  assumes: x + n inside the table; l_(x+n) > 0; 0 < i < 1
```

In the worked example, lx = [100000, 99500, 98900, 98200, 97300], x0 = 40, x = 40, n = 4, i = 0.05 go through the equation above and give nEx = 0.800489508, accumulation = 1.249235612. It applies only when x + n inside the table; l_(x+n) > 0; 0 < i < 1: outside that it says what is wrong instead of returning a number.

Survival to a fractional age (UDD, constant force, Balducci) — `act_fractional_age`

The probability of surviving (or dying within) a fraction s of a year from integer age x, and the force of mortality at x+s, under a uniform distribution of deaths, a constant force of mortality or the Balducci assumption.

$$\text{UDD: } {}_sp_x = 1 - sq_x,\ \mu_{x+s} = \frac{q_x}{1 - sq_x};\quad \text{CF: } {}_sp_x = p_x^s;\quad \text{Balducci: } {}_sp_x = \frac{p_x}{1 - (1-s)q_x}$$

symbol		meaning	SI unit
`q`	in	one-year death probability q_x	—
`s`	in	fraction of the year (0 < s < 1)	—
`assumption`	in	fractional-age assumption	—
`spx`	out	probability of surviving to age x+s	—
`sqx`	out	probability of dying before age x+s	—
`mu`	out	force of mortality at age x+s	—

Applies when: 0 < q < 1; 0 < s < 1.

```
$ actuarial-science solve act_fractional_age q=0.1 s=0.5 assumption=udd
spx = 0.95
sqx = 0.05
mu = 0.1052631579
  by: Survival to a fractional age (UDD, constant force, Balducci)  (UDD: s_p_x = 1 - s q, mu =
q/(1 - s q); constant force: (1-q)^s, mu = -ln(1-q); Balducci: (1-q)/(1-(1-s)q), mu = q/(1-(1-s)q))
  q = 0.1
  s = 0.5
  assumption = udd
  assumes: 0 < q < 1; 0 < s < 1
```

In the worked example, q = 0.1, s = 0.5, assumption = udd go through the equation above and give spx = 0.95, sqx = 0.05, mu = 0.1052631579. It applies only when 0 < q < 1; 0 < s < 1: outside that it says what is wrong instead of returning a number.

Temporary (n-year) expectation of life — `act_temporary_expectation`

The expected number of whole years (curtate) and of years (complete, UDD) lived within the next n years by a life aged x.

$$e_{x:\overline{n}|} = \sum_{k=1}^{n} {}_kp_x, \quad \mathring{e}_{x:\overline{n}|} = \sum_{k=0}^{n-1} \frac{{}_kp_x + {}_{k+1}p_x}{2}$$

symbol		meaning	SI unit
`lx`	in	survivors at consecutive integer ages from x0 (a list)	—
`x0`	in	age of the first entry of lx	—
`x`	in	age of the life	—
`n`	in	term in years	—
`e_curtate`	out	curtate n-year temporary expectation e_(x:n)	—
`e_complete`	out	complete n-year temporary expectation under UDD	—

Applies when: x + n inside the table; n >= 1.

```
$ actuarial-science solve act_temporary_expectation "lx=[1000, 900, 700, 400, 0]" x0=95 x=95 n=2
e_curtate = 1.6
e_complete = 1.75
  by: Temporary (n-year) expectation of life  (e_(x:n) = sum_(k=1..n) k_p_x; e°_(x:n) =
sum_(k=0..n-1) (k_p_x + (k+1)_p_x)/2)
  assumes: x + n inside the table; n >= 1
```

In the worked example, lx = [1000, 900, 700, 400, 0], x0 = 95, x = 95, n = 2 go through the equation above and give e_curtate = 1.6, e_complete = 1.75. It applies only when x + n inside the table; n >= 1: outside that it says what is wrong instead of returning a number.

Loss models (16)

Compound Poisson mean, variance and skewness — `act_compound_poisson_moments`

The mean, variance and coefficient of skewness of aggregate losses when the claim count is Poisson(lambda), from the first three raw moments of the claim severity.

$$\kappa_n(S) = \lambda\, E[X^n], \quad \gamma_S = \frac{\lambda E[X^3]}{(\lambda E[X^2])^{3/2}}$$

symbol		meaning	SI unit
`lam`	in	Poisson mean number of claims lambda	—
`m1`	in	E[X]	—
`m2`	in	E[X^2]	—
`m3`	in	E[X^3]	—
`mean`	out	lambda E[X]	—
`var`	out	lambda E[X^2]	—
`skew`	out	lambda E[X^3] / (lambda E[X^2])^(3/2)	—

Applies when: lambda > 0; moments positive; E[X^2] >= E[X]^2.

```
$ actuarial-science solve act_compound_poisson_moments lam=3 m1=2 m2=6 m3=24
mean = 6
var = 18
skew = 0.9428090416
  by: Compound Poisson mean, variance and skewness  (cumulants of a compound Poisson sum: kappa_n =
lambda E[X^n])
  lam = 3
  m1 = 2
  m2 = 6
  m3 = 24
  assumes: lambda > 0; moments positive; E[X^2] >= E[X]^2
```

In the worked example, lam = 3, m1 = 2, m2 = 6, m3 = 24 go through the equation above and give mean = 6, var = 18, skew = 0.9428090416. It applies only when lambda > 0; moments positive; E[X^2] >= E[X]^2: outside that it says what is wrong instead of returning a number.

Deductible, policy limit, coinsurance and inflation with exponential losses — `act_exponential_coverage_modifications`

The expected insurer payment per loss and per payment for exponential losses (mean theta) under an ordinary deductible, a policy limit (maximum covered loss), coinsurance and uniform inflation.

$$E[Y^L] = c(1+r)\left[E\left[X \wedge \tfrac{u}{1+r}\right] - E\left[X \wedge \tfrac{d}{1+r}\right]\right], \quad E[Y^P] = \frac{E[Y^L]}{S_X\left(\tfrac{d}{1+r}\right)}$$

symbol		meaning	SI unit
`theta`	in	mean loss theta before inflation	—
`d`	in	ordinary deductible	—
`u`	in	policy limit (maximum covered loss)	—
`c`	in	coinsurance fraction paid by the insurer (80% or 0.8)	—
`r`	in	uniform loss inflation (0 for none)	—
`per_loss`	out	expected payment per loss	—
`per_payment`	out	expected payment per payment	—
`max_payment`	out	largest possible payment c (u - d)	—

Applies when: theta > 0; 0 <= d < u; 0 < c <= 1; 0 <= r < 1.

```
$ actuarial-science solve act_exponential_coverage_modifications theta=1000 d=200 u=3000 c=0.8
r=0.1
per_loss = 676.1528531
per_payment = 810.9750963
max_payment = 2240
  by: Deductible, policy limit, coinsurance and inflation with exponential losses  (per loss = c (1
+ r) [E[X ∧ u/(1+r)] - E[X ∧ d/(1+r)]]; per payment = per loss / S(d/(1+r)))
  theta = 1000
  d = 200
  u = 3000
  c = 0.8
  r = 0.1
  assumes: theta > 0; 0 <= d < u; 0 < c <= 1; 0 <= r < 1
```

In the worked example, theta = 1000, d = 200, u = 3000, c = 0.8, r = 0.1 go through the equation above and give per_loss = 676.1528531, per_payment = 810.9750963, max_payment = 2240. It applies only when theta > 0; 0 <= d < u; 0 < c <= 1; 0 <= r < 1: outside that it says what is wrong instead of returning a number.

Deductible, policy limit, coinsurance and inflation with Pareto losses — `act_pareto_coverage_modifications`

The expected insurer payment per loss and per payment for two-parameter Pareto losses under an ordinary deductible, a policy limit (maximum covered loss), coinsurance and uniform inflation.

$$E[Y^L] = c(1+r)\left[E\left[X \wedge \tfrac{u}{1+r}\right] - E\left[X \wedge \tfrac{d}{1+r}\right]\right], \quad E[Y^P] = \frac{E[Y^L]}{S_X\left(\tfrac{d}{1+r}\right)}$$

symbol		meaning	SI unit
`alpha`	in	Pareto shape alpha (> 1)	—
`theta`	in	Pareto scale theta before inflation	—
`d`	in	ordinary deductible	—
`u`	in	policy limit (maximum covered loss)	—
`c`	in	coinsurance fraction paid by the insurer (80% or 0.8)	—
`r`	in	uniform loss inflation (0 for none)	—
`per_loss`	out	expected payment per loss	—
`per_payment`	out	expected payment per payment	—
`max_payment`	out	largest possible payment c (u - d)	—

Applies when: alpha > 1; theta > 0; 0 <= d < u; 0 < c <= 1; 0 <= r < 1.

```
$ actuarial-science solve act_pareto_coverage_modifications alpha=3 theta=2000 d=500 u=5000 c=0.8
r=0.05
per_loss = 474.5027461
per_payment = 900.535608
max_payment = 3600
  by: Deductible, policy limit, coinsurance and inflation with Pareto losses  (per loss = c (1 + r)
[E[X ∧ u/(1+r)] - E[X ∧ d/(1+r)]]; per payment = per loss / S(d/(1+r)))
  alpha = 3
  theta = 2000
  d = 500
  u = 5000
  c = 0.8
  r = 0.05
  assumes: alpha > 1; theta > 0; 0 <= d < u; 0 < c <= 1; 0 <= r < 1
```

In the worked example, alpha = 3, theta = 2000, d = 500, u = 5000, c = 0.8, r = 0.05 go through the equation above and give per_loss = 474.5027461, per_payment = 900.535608, max_payment = 3600. It applies only when alpha > 1; theta > 0; 0 <= d < u; 0 < c <= 1; 0 <= r < 1: outside that it says what is wrong instead of returning a number.

Individual risk model: aggregate mean and variance — `act_individual_risk_model`

The mean and variance of total claims, and the probability of no claim, for a portfolio of independent policies each paying a fixed benefit b_i with probability q_i.

$$E[S] = \sum_i q_i b_i, \quad \mathrm{Var}(S) = \sum_i q_i (1 - q_i)\, b_i^2$$

symbol		meaning	SI unit
`q`	in	claim probabilities of the policies (a list)	—
`b`	in	benefit amounts of the policies (a list)	—
`mean`	out	expected total claims sum q b	—
`var`	out	variance sum q (1-q) b^2	—
`prob_no_claim`	out	probability of no claim at all	—

Applies when: one benefit per probability; 0 <= q <= 1; b > 0.

```
$ actuarial-science solve act_individual_risk_model "q=[0.01, 0.02, 0.05]" "b=[10000, 5000, 2000]"
mean = 300
var = 1670000
prob_no_claim = 0.92169
  by: Individual risk model: aggregate mean and variance  (E[S] = sum q_i b_i; Var S = sum q_i (1 -
q_i) b_i^2; P(S = 0) = prod (1 - q_i))
  assumes: one benefit per probability; 0 <= q <= 1; b > 0
```

In the worked example, q = [0.01, 0.02, 0.05], b = [10000, 5000, 2000] go through the equation above and give mean = 300, var = 1670000, prob_no_claim = 0.92169. It applies only when one benefit per probability; 0 <= q <= 1; b > 0: outside that it says what is wrong instead of returning a number.

Insurer payment for one loss (deductible, limit, coinsurance) — `act_single_loss_payment`

The amount the insurer pays and the amount the insured retains for a single loss x under an ordinary deductible d, a policy limit u (maximum covered loss) and coinsurance c.

$$Y = c(\min(X, u) - d)_+$$

symbol		meaning	SI unit
`x`	in	loss amount	—
`d`	in	ordinary deductible	—
`u`	in	policy limit (maximum covered loss)	—
`c`	in	coinsurance fraction paid by the insurer (80% or 0.8)	—
`payment`	out	insurer payment c (min(x, u) - d)+	—
`retained`	out	amount retained by the insured	—

Applies when: x, d >= 0; u > d; 0 < c <= 1.

```
$ actuarial-science solve act_single_loss_payment x=5000 d=500 u=4000 c=0.8
payment = 2800
retained = 2200
  by: Insurer payment for one loss (deductible, limit, coinsurance)  (payment = c x (min(x, u) -
d)+)
  x = 5000
  d = 500
  u = 4000
  c = 0.8
  assumes: x, d >= 0; u > d; 0 < c <= 1
```

In the worked example, x = 5000, d = 500, u = 4000, c = 0.8 go through the equation above and give payment = 2800, retained = 2200. It applies only when x, d >= 0; u > d; 0 < c <= 1: outside that it says what is wrong instead of returning a number.

Mean and variance of aggregate losses (compound distribution) — `act_compound_distribution_moments`

The mean and variance of aggregate losses S = X_1 + ... + X_N from the mean and variance of the claim count N and of the claim severity X (independent, identically distributed severities).

$$E[S] = E[N]\,E[X], \quad \operatorname{Var}(S) = E[N]\operatorname{Var}(X) + \operatorname{Var}(N)\,E[X]^2$$

symbol		meaning	SI unit
`EN`	in	mean number of claims E[N]	—
`VarN`	in	variance of the number of claims	—
`EX`	in	mean claim severity E[X]	—
`VarX`	in	variance of the claim severity	—
`ES`	out	mean aggregate loss E[N] E[X]	—
`VarS`	out	variance E[N] Var X + Var N E[X]^2	—

Applies when: moments non-negative; N and the X's independent.

```
$ actuarial-science solve act_compound_distribution_moments EN=3 VarN=3 EX=1000 VarX=250000
ES = 3000
VarS = 3750000
  by: Mean and variance of aggregate losses (compound distribution)  (E[S] = E[N] E[X]; Var S =
E[N] Var X + Var N (E[X])^2)
  EN = 3
  VarN = 3
  EX = 1000
  VarX = 250000
  assumes: moments non-negative; N and the X's independent
```

In the worked example, EN = 3, VarN = 3, EX = 1000, VarX = 250000 go through the equation above and give ES = 3000, VarS = 3750000. It applies only when moments non-negative; N and the X's independent: outside that it says what is wrong instead of returning a number.

Negative binomial claim frequency probabilities — `act_negative_binomial_frequency`

The probability of exactly k claims and of at most k claims, with the mean and variance, for a negative binomial claim count with parameters r and beta (mean r beta).

$$P(N=k)=\binom{r+k-1}{k}\left(\frac{1}{1+\beta}\right)^{r}\left(\frac{\beta}{1+\beta}\right)^{k}$$

symbol		meaning	SI unit
`r`	in	negative binomial r	—
`beta`	in	negative binomial beta	—
`k`	in	number of claims	—
`pmf`	out	P(N = k)	—
`cdf`	out	P(N <= k)	—
`mean`	out	mean r beta	—
`var`	out	variance r beta (1 + beta)	—

Applies when: r, beta > 0; k >= 0.

```
$ actuarial-science solve act_negative_binomial_frequency r=2 beta=1.5 k=3
pmf = 0.13824
cdf = 0.66304
mean = 3
var = 7.5
  by: Negative binomial claim frequency probabilities  (P(N = k) = Gamma(r+k)/(Gamma(r) k!)
(1/(1+beta))^r (beta/(1+beta))^k)
  r = 2
  beta = 1.5
  assumes: r, beta > 0; k >= 0
```

In the worked example, r = 2, beta = 1.5, k = 3 go through the equation above and give pmf = 0.13824, cdf = 0.66304, mean = 3, var = 7.5. It applies only when r, beta > 0; k >= 0: outside that it says what is wrong instead of returning a number.

Normal approximation to aggregate losses — `act_normal_approximation_aggregate`

The probability that aggregate losses exceed an amount s under the normal approximation with the given mean and variance.

$$P(S > s) \approx 1 - \Phi\left(\frac{s - E[S]}{\sqrt{\mathrm{Var}(S)}}\right)$$

symbol		meaning	SI unit
`mean`	in	mean of aggregate losses	—
`var`	in	variance of aggregate losses	—
`s`	in	threshold amount	—
`prob_exceed`	out	P(S > s) by the normal approximation	—
`z`	out	standardized threshold (s - mean)/sd	—

Applies when: variance > 0.

```
$ actuarial-science solve act_normal_approximation_aggregate mean=3000 var=1000000 s=5000
prob_exceed = 0.02275013195
z = 2
  by: Normal approximation to aggregate losses  (P(S > s) = 1 - Phi((s - E[S])/sd(S)))
  mean = 3000
  var = 1000000
  s = 5000
  assumes: variance > 0
```

In the worked example, mean = 3000, var = 1000000, s = 5000 go through the equation above and give prob_exceed = 0.02275013195, z = 2. It applies only when variance > 0: outside that it says what is wrong instead of returning a number.

Ordinary deductible with exponential losses — `act_exponential_deductible`

The expected payment per loss and per payment, the limited expected value and the loss elimination ratio of an ordinary deductible d when losses are exponential with mean theta.

$$E[X \wedge d] = \theta\left(1 - e^{-d/\theta}\right), \quad E[(X - d)_+] = \theta e^{-d/\theta}, \quad e(d) = \theta$$

symbol		meaning	SI unit
`theta`	in	mean loss theta	—
`d`	in	ordinary deductible	—
`mean`	out	mean loss E[X]	—
`lev_d`	out	limited expected value E[X ∧ d]	—
`per_loss`	out	expected payment per loss E[(X - d)+]	—
`per_payment`	out	expected payment per payment (mean excess loss e(d))	—
`LER`	out	loss elimination ratio E[X ∧ d]/E[X]	—

Applies when: theta > 0; d >= 0.

```
$ actuarial-science solve act_exponential_deductible theta=1000 d=500
mean = 1000
lev_d = 393.4693403
per_loss = 606.5306597
per_payment = 1000
LER = 0.3934693403
  by: Ordinary deductible with exponential losses  (per loss = E[X] - E[X ∧ d]; per payment = per
loss / S(d); LER = E[X ∧ d]/E[X]; E[X ∧ d] = theta (1 - e^(-d/theta)))
  theta = 1000
  d = 500
  assumes: theta > 0; d >= 0
```

In the worked example, theta = 1000, d = 500 go through the equation above and give mean = 1000, lev_d = 393.4693403, per_loss = 606.5306597, per_payment = 1000. It applies only when theta > 0; d >= 0: outside that it says what is wrong instead of returning a number.

Ordinary deductible with lognormal losses — `act_lognormal_deductible`

The expected payment per loss and per payment, the limited expected value and the loss elimination ratio of an ordinary deductible d when losses are lognormal (mu, sigma of the log of the loss).

$$E[X \wedge d] = e^{\mu+\sigma^2/2}\,\Phi\left(\frac{\ln d - \mu - \sigma^2}{\sigma}\right) + d\left[1 - \Phi\left(\frac{\ln d - \mu}{\sigma}\right)\right]$$

symbol		meaning	SI unit
`mu`	in	mean of the log of the loss	—
`sigma`	in	standard deviation of the log of the loss	—
`d`	in	ordinary deductible	—
`mean`	out	mean loss E[X]	—
`lev_d`	out	limited expected value E[X ∧ d]	—
`per_loss`	out	expected payment per loss E[(X - d)+]	—
`per_payment`	out	expected payment per payment (mean excess loss e(d))	—
`LER`	out	loss elimination ratio E[X ∧ d]/E[X]	—

Applies when: sigma > 0; d >= 0.

```
$ actuarial-science solve act_lognormal_deductible mu=7 sigma=1 d=1000
mean = 1808.042414
lev_d = 785.1059202
per_loss = 1022.936494
per_payment = 1905.803319
LER = 0.4342298134
  by: Ordinary deductible with lognormal losses  (per loss = E[X] - E[X ∧ d]; per payment = per
loss / S(d); LER = E[X ∧ d]/E[X]; E[X ∧ d] = e^(mu + sigma^2/2) Phi((ln d - mu - sigma^2)/sigma) +
d [1 - Phi((ln d - mu)/sigma)])
  mu = 7
  sigma = 1
  d = 1000
  assumes: sigma > 0; d >= 0
```

In the worked example, mu = 7, sigma = 1, d = 1000 go through the equation above and give mean = 1808.042414, lev_d = 785.1059202, per_loss = 1022.936494, per_payment = 1905.803319. It applies only when sigma > 0; d >= 0: outside that it says what is wrong instead of returning a number.

Ordinary deductible with Pareto losses — `act_pareto_deductible`

The expected payment per loss and per payment, the limited expected value and the loss elimination ratio of an ordinary deductible d when losses follow a two-parameter Pareto distribution (alpha, theta).

$$E[X \wedge d] = \frac{\theta}{\alpha - 1}\left[1 - \left(\frac{\theta}{\theta + d}\right)^{\alpha - 1}\right], \quad e(d) = \frac{\theta + d}{\alpha - 1}$$

symbol		meaning	SI unit
`alpha`	in	Pareto shape alpha (> 1)	—
`theta`	in	Pareto scale theta	—
`d`	in	ordinary deductible	—
`mean`	out	mean loss E[X]	—
`lev_d`	out	limited expected value E[X ∧ d]	—
`per_loss`	out	expected payment per loss E[(X - d)+]	—
`per_payment`	out	expected payment per payment (mean excess loss e(d))	—
`LER`	out	loss elimination ratio E[X ∧ d]/E[X]	—

Applies when: alpha > 1; theta > 0; d >= 0.

```
$ actuarial-science solve act_pareto_deductible alpha=3 theta=2000 d=500
mean = 1000
lev_d = 360
per_loss = 640
per_payment = 1250
LER = 0.36
  by: Ordinary deductible with Pareto losses  (per loss = E[X] - E[X ∧ d]; per payment = per loss /
S(d); LER = E[X ∧ d]/E[X]; E[X ∧ d] = theta/(alpha-1) [1 - (theta/(theta+d))^(alpha-1)])
  alpha = 3
  theta = 2000
  d = 500
  assumes: alpha > 1; theta > 0; d >= 0
```

In the worked example, alpha = 3, theta = 2000, d = 500 go through the equation above and give mean = 1000, lev_d = 360, per_loss = 640, per_payment = 1250. It applies only when alpha > 1; theta > 0; d >= 0: outside that it says what is wrong instead of returning a number.

Panjer recursion for a compound negative binomial distribution — `act_panjer_negative_binomial`

The probabilities of aggregate losses 0, 1, ..., smax units for a negative binomial (r, beta) number of claims with claim severities on 0, 1, 2, ... units, by Panjer's recursion, and P(S <= smax).

$$g_s = \frac{1}{1 - a f_0} \sum_{j=1}^{s} \left(a + \frac{bj}{s} \right) f_j \, g_{s-j}, \quad a = \frac{\beta}{1+\beta}, \; b = (r-1)a$$

symbol		meaning	SI unit
`r`	in	negative binomial r	—
`beta`	in	negative binomial beta	—
`f`	in	severity probabilities of 0, 1, 2, ... units (a list adding to 1)	—
`smax`	in	largest aggregate amount wanted	—
`g`	out	P(S = s) for s = 0..smax (a list)	—
`F_smax`	out	P(S <= smax)	—

Applies when: r, beta > 0; severity probabilities add to 1; smax <= 300.

```
$ actuarial-science solve act_panjer_negative_binomial r=2 beta=1 "f=[0, 0.5, 0.3, 0.2]" smax=5
g = [0.25,0.125,0.121875,0.121875,0.0873828125,0.07130859375]
F_smax = 0.7774414063
  by: Panjer recursion for a compound negative binomial distribution  ((a, b, 0) class: a =
beta/(1+beta), b = (r-1) a; g_0 = (1 + beta(1 - f_0))^(-r); g_s = [sum (a + b j/s) f_j g_(s-j)]/(1
- a f_0))
  r = 2
  beta = 1
  assumes: r, beta > 0; severity probabilities add to 1; smax <= 300
```

In the worked example, r = 2, beta = 1, f = [0, 0.5, 0.3, 0.2], smax = 5 go through the equation above and give g = [0.25, 0.125, 0.121875, 0.121875, 0.0873828125, 0.07130859375], F_smax = 0.7774414063. It applies only when r, beta > 0; severity probabilities add to 1; smax <= 300: outside that it says what is wrong instead of returning a number.

Panjer recursion for a compound Poisson distribution — `act_panjer_poisson`

The probabilities of aggregate losses 0, 1, ..., smax (in monetary units) for a Poisson(lambda) number of claims with claim severities on 0, 1, 2, ... units, by Panjer's recursion, and P(S <= smax).

$$g_0 = e^{-\lambda(1-f_0)}, \quad g_s = \frac{\lambda}{s}\sum_{j=1}^{s} j\, f_j\, g_{s-j}$$

symbol		meaning	SI unit
`lam`	in	Poisson mean number of claims lambda	—
`f`	in	severity probabilities of 0, 1, 2, ... units (a list adding to 1)	—
`smax`	in	largest aggregate amount wanted	—
`g`	out	P(S = s) for s = 0..smax (a list)	—
`F_smax`	out	P(S <= smax)	—

Applies when: 0 < lambda <= 200; severity probabilities add to 1; smax <= 300.

```
$ actuarial-science solve act_panjer_poisson lam=2 "f=[0, 0.5, 0.3, 0.2]" smax=5
g = [0.135335283237,0.135335283237,0.14886881156,0.157891163776,0.124734019383,0.0985691979573]
F_smax = 0.8007337592
  by: Panjer recursion for a compound Poisson distribution  (g_0 = e^(-lambda(1 - f_0)); g_s =
(1/s) sum_(j=1..s) lambda j f_j g_(s-j))
  lam = 2
  assumes: 0 < lambda <= 200; severity probabilities add to 1; smax <= 300
```

In the worked example, lam = 2, f = [0, 0.5, 0.3, 0.2], smax = 5 go through the equation above and give g = [0.1353352832, 0.1353352832, 0.1488688116, 0.1578911638, 0.1247340194, 0.09856919796], F_smax = 0.8007337592. It applies only when 0 < lambda <= 200; severity probabilities add to 1; smax <= 300: outside that it says what is wrong instead of returning a number.

Poisson claim frequency probabilities — `act_poisson_frequency`

The probability of exactly k claims and of at most k claims when the number of claims is Poisson with mean lambda.

$$P(N = k) = \frac{e^{-\lambda}\lambda^{k}}{k!}$$

symbol		meaning	SI unit
`lam`	in	Poisson mean number of claims lambda	—
`k`	in	number of claims	—
`pmf`	out	P(N = k)	—
`cdf`	out	P(N <= k)	—
`mean`	out	mean number of claims	—
`var`	out	variance of the number of claims	—

Applies when: 0 < lambda <= 500; k >= 0.

```
$ actuarial-science solve act_poisson_frequency lam=2 k=3
pmf = 0.1804470443
cdf = 0.8571234605
mean = 2
var = 2
  by: Poisson claim frequency probabilities  (P(N = k) = e^(-lambda) lambda^k / k!)
  lam = 2
  assumes: 0 < lambda <= 500; k >= 0
```

In the worked example, lam = 2, k = 3 go through the equation above and give pmf = 0.1804470443, cdf = 0.8571234605, mean = 2, var = 2. It applies only when 0 < lambda <= 500; k >= 0: outside that it says what is wrong instead of returning a number.

Premium principles: expected value, variance, standard deviation — `act_premium_principles`

The premium for a risk with a given mean and variance of loss by the expected value principle (1 + theta) E[X], the variance principle E[X] + theta Var X or the standard deviation principle E[X] + theta sd(X).

$$\pi = (1+\theta)E[X], \quad \pi = E[X] + \theta\,\mathrm{Var}(X), \quad \pi = E[X] + \theta\sqrt{\mathrm{Var}(X)}$$

symbol		meaning	SI unit
`mean`	in	expected loss	—
`var`	in	variance of the loss	—
`theta`	in	loading factor	—
`principle`	in	premium principle	—
`premium`	out	premium	—
`loading`	out	risk loading added to the expected loss	—

Applies when: mean, variance, loading >= 0.

```
$ actuarial-science solve act_premium_principles mean=1000 var=250000 theta=0.2
principle=expected_value
premium = 1200
loading = 200
  by: Premium principles: expected value, variance, standard deviation  (expected value: (1 +
theta) E[X]; variance: E[X] + theta Var X; standard deviation: E[X] + theta sd(X))
  mean = 1000
  var = 250000
  theta = 0.2
  principle = expected_value
  assumes: mean, variance, loading >= 0
```

In the worked example, mean = 1000, var = 250000, theta = 0.2, principle = expected_value go through the equation above and give premium = 1200, loading = 200. It applies only when mean, variance, loading >= 0: outside that it says what is wrong instead of returning a number.

Stop-loss premium for a compound Poisson distribution — `act_stop_loss_premium`

The net stop-loss premium E[(S - d)+] for a retention of d units, with E[S] and the limited expectation E[S ∧ d], when S is compound Poisson with claim severities on 0, 1, 2, ... units.

$$E[(S-d)_+] = E[S] - \sum_{s=0}^{d-1}(1 - F_S(s))$$

symbol		meaning	SI unit
`lam`	in	Poisson mean number of claims lambda	—
`f`	in	severity probabilities of 0, 1, 2, ... units (a list adding to 1)	—
`d`	in	retention (whole number of units)	—
`premium`	out	net stop-loss premium E[(S - d)+]	—
`ES`	out	expected aggregate loss E[S]	—
`limited`	out	limited expectation E[S ∧ d]	—

Applies when: 0 < lambda <= 200; severity probabilities add to 1; whole-number retention.

```
$ actuarial-science solve act_stop_loss_premium lam=2 "f=[0, 0.5, 0.3, 0.2]" d=3
premium = 1.225545228
ES = 3.4
limited = 2.174454772
  by: Stop-loss premium for a compound Poisson distribution  (E[(S - d)+] = E[S] - sum_(s=0..d-1)
P(S > s), with P(S = s) from Panjer's recursion)
  lam = 2
  assumes: 0 < lambda <= 200; severity probabilities add to 1; whole-number retention
```

In the worked example, lam = 2, f = [0, 0.5, 0.3, 0.2], d = 3 go through the equation above and give premium = 1.225545228, ES = 3.4, limited = 2.174454772. It applies only when 0 < lambda <= 200; severity probabilities add to 1; whole-number retention: outside that it says what is wrong instead of returning a number.

Loss reserving (7)

Bornhuetter-Ferguson reserve — `act_bornhuetter_ferguson`

The Bornhuetter-Ferguson reserve and ultimate losses of each origin year from a cumulative loss triangle, the earned premiums and an expected loss ratio: the unreported part (1 - 1/CDF) of the expected losses, with chain-ladder volume-weighted factors.

$$R_i^{BF} = P_i \cdot \mathrm{ELR}\left(1 - \frac{1}{F_{N-i}}\right), \qquad \hat{U}_i = C_{i,N-i} + R_i^{BF}$$

symbol		meaning	SI unit
`T`	in	cumulative loss triangle: origin years in rows, development ages in columns, unknown cells written as 0	—
`P`	in	earned premium of each origin year (a list)	—
`elr`	in	expected (a priori) loss ratio (70% or 0.7)	—

symbol		meaning	SI unit
`ultimate`	out	ultimate losses of each origin year (a list)	—
`reserve`	out	reserve of each origin year (a list)	—
`total_reserve`	out	total reserve	—

Applies when: square triangle; one positive premium per origin year; 0 < ELR <= 500%.

```
$ actuarial-science solve act_bornhuetter_ferguson "T=[[100, 150, 175], [110, 168, 0], [120, 0,
0]]" "P=[200, 210, 220]" elr=0.8
ultimate = [175.0,192.0,196.377358491]
reserve = [0.0,24.0,76.3773584906]
total_reserve = 100.3773585
  by: Bornhuetter-Ferguson reserve  (reserve_i = P_i ELR (1 - 1/CDF_i); ultimate_i = latest_i +
reserve_i)
  elr = 0.8
  assumes: square triangle; one positive premium per origin year; 0 < ELR <= 500%
```

In the worked example, T = [100 150 175; 110 168 0; 120 0 0], P = [200, 210, 220], elr = 0.8 go through the equation above and give ultimate = [175, 192, 196.3773585], reserve = [0, 24, 76.37735849], total_reserve = 100.3773585. It applies only when square triangle; one positive premium per origin year; 0 < ELR <= 500%: outside that it says what is wrong instead of returning a number.

Cape Cod (Stanard-Buhlmann) reserve — `act_cape_cod`

The Cape Cod expected loss ratio (reported losses over used-up premium), and the resulting reserves and ultimate losses of each origin year, from a cumulative loss triangle and the earned premiums.

$$\mathrm{ELR}_{CC} = \frac{\sum_i C_{i,N-i}}{\sum_i P_i / F_{N-i}}, \qquad R_i = P_i \, \mathrm{ELR}_{CC} \left(1 - \frac{1}{F_{N-i}}\right)$$

symbol		meaning	SI unit
`T`	in	cumulative loss triangle: origin years in rows, development ages in columns, unknown cells written as 0	—
`P`	in	earned premium of each origin year (a list)	—
`elr`	out	Cape Cod expected loss ratio	—
`ultimate`	out	ultimate losses of each origin year (a list)	—
`reserve`	out	reserve of each origin year (a list)	—
`total_reserve`	out	total reserve	—

Applies when: square triangle; one positive premium per origin year; no trend, no decay.

```
$ actuarial-science solve act_cape_cod "T=[[100, 150, 175], [110, 168, 0], [120, 0, 0]]" "P=[200,
210, 220]"
elr = 0.9176888556
ultimate = [175.0,195.530665669,207.613313388]
reserve = [0.0,27.5306656694,87.6133133882]
total_reserve = 115.1439791
  by: Cape Cod (Stanard-Buhlmann) reserve  (ELR = sum latest_i / sum (P_i / CDF_i); reserve_i = P_i
ELR (1 - 1/CDF_i))
  assumes: square triangle; one positive premium per origin year; no trend, no decay
```

In the worked example, T = [100 150 175; 110 168 0; 120 0 0], P = [200, 210, 220] go through the equation above and give elr = 0.9176888556, ultimate = [175, 195.5306657, 207.6133134], reserve = [0, 27.53066567, 87.61331339], total_reserve = 115.1439791. It applies only when square triangle; one positive premium per origin year; no trend, no decay: outside that it says what is wrong instead of returning a number.

Chain-ladder age-to-age and cumulative development factors — `act_chain_ladder_factors`

The volume-weighted age-to-age development factors of a cumulative loss triangle and the cumulative factors to ultimate from each development age (no tail factor).

$$\hat{f}_j = \frac{\sum_i C_{i,j+1}}{\sum_i C_{i,j}}, \qquad F_j = \prod_{k \geq j} \hat{f}_k$$

symbol		meaning	SI unit
`T`	in	cumulative loss triangle: origin years in rows, development ages in columns, unknown cells written as 0	—
`ldf`	out	age-to-age factors f_1 ... f_(N-1) (a list)	—
`cdf`	out	cumulative development factors to ultimate from each age, last = 1 (a list)	—

Applies when: square triangle with N >= 2; known cells positive, unknown cells 0; no tail factor.

```
$ actuarial-science solve act_chain_ladder_factors "T=[[100, 150, 175], [110, 168, 0], [120, 0,
0]]"
ldf = [1.51428571429,1.16666666667]
cdf = [1.76666666667,1.16666666667,1.0]
  by: Chain-ladder age-to-age and cumulative development factors  (f_j = sum_i C_(i,j+1) / sum_i
C_(i,j) over origins with both ages known; CDF_j = product of f from age j on)
  assumes: square triangle with N >= 2; known cells positive, unknown cells 0; no tail factor
```

In the worked example, T = [100 150 175; 110 168 0; 120 0 0] go through the equation above and give ldf = [1.514285714, 1.166666667], cdf = [1.766666667, 1.166666667, 1]. It applies only when square triangle with N >= 2; known cells positive, unknown cells 0; no tail factor: outside that it says what is wrong instead of returning a number.

Chain-ladder ultimate losses and reserves — `act_chain_ladder_reserve`

The ultimate losses, the reserve (ultimate less reported to date) of each origin year and the total reserve of a cumulative loss triangle by the volume-weighted chain-ladder method (no tail factor).

$$\hat{C}_{i,N} = C_{i,N-i} \prod_{k \geq N-i} \hat{f}_k, \qquad R_i = \hat{C}_{i,N} - C_{i,N-i}$$

symbol		meaning	SI unit
`T`	in	cumulative loss triangle: origin years in rows, development ages in columns, unknown cells written as 0	—
`ultimate`	out	ultimate losses of each origin year (a list)	—
`reserve`	out	reserve of each origin year (a list)	—
`total_reserve`	out	total reserve	—

Applies when: square triangle with N >= 2; known cells positive, unknown cells 0; no tail factor.

```
$ actuarial-science solve act_chain_ladder_reserve "T=[[100, 150, 175], [110, 168, 0], [120, 0,
0]]"
ultimate = [175.0,196.0,212.0]
reserve = [0.0,28.0,92.0]
total_reserve = 120
  by: Chain-ladder ultimate losses and reserves  (ultimate_i = latest_i x CDF at its age; reserve_i
= ultimate_i - latest_i)
  assumes: square triangle with N >= 2; known cells positive, unknown cells 0; no tail factor
```

In the worked example, T = [100 150 175; 110 168 0; 120 0 0] go through the equation above and give ultimate = [175, 196, 212], reserve = [0, 28, 92], total_reserve = 120. It applies only when square triangle with N >= 2; known cells positive, unknown cells 0; no tail factor: outside that it says what is wrong instead of returning a number.

Chain-ladder with simple-average development factors — `act_chain_ladder_simple_average`

The simple-average age-to-age factors (mean of the individual link ratios), the ultimate losses of each origin year and the total reserve of a cumulative loss triangle (no tail factor).

$$\hat{f}_j = \frac{1}{n_j} \sum_i \frac{C_{i,j+1}}{C_{i,j}}$$

symbol		meaning	SI unit
`T`	in	cumulative loss triangle: origin years in rows, development ages in columns, unknown cells written as 0	—
`ldf`	out	simple-average age-to-age factors (a list)	—
`ultimate`	out	ultimate losses of each origin year (a list)	—
`total_reserve`	out	total reserve	—

Applies when: square triangle with N >= 2; known cells positive, unknown cells 0; no tail factor.

```
$ actuarial-science solve act_chain_ladder_simple_average "T=[[100, 150, 175], [110, 168, 0], [120,
0, 0]]"
ldf = [1.51363636364,1.16666666667]
ultimate = [175.0,196.0,211.909090909]
total_reserve = 119.9090909
  by: Chain-ladder with simple-average development factors  (f_j = mean over origins of
C_(i,j+1)/C_(i,j); ultimate_i = latest_i x product of f)
  assumes: square triangle with N >= 2; known cells positive, unknown cells 0; no tail factor
```

In the worked example, T = [100 150 175; 110 168 0; 120 0 0] go through the equation above and give ldf = [1.513636364, 1.166666667], ultimate = [175, 196, 211.9090909], total_reserve = 119.9090909. It applies only when square triangle with N >= 2; known cells positive, unknown cells 0; no tail factor: outside that it says what is wrong instead of returning a number.

Expected claims reserving method — `act_expected_claims_method`

The ultimate losses (premium x expected loss ratio), the reserve (ultimate less paid or reported to date) of each origin year and the total reserve by the expected claims method.

$$\hat{U}_i = P_i \cdot \mathrm{ELR}, \qquad R_i = \hat{U}_i - C_i$$

symbol		meaning	SI unit
`P`	in	earned premium of each origin year (a list)	—
`elr`	in	expected loss ratio (65% or 0.65)	—
`paid`	in	losses paid (or reported) to date of each origin year (a list)	—
`ultimate`	out	expected ultimate losses (a list)	—
`reserve`	out	reserve of each origin year (a list)	—
`total_reserve`	out	total reserve	—

Applies when: one paid amount per premium; 0 < ELR <= 500%.

```
$ actuarial-science solve act_expected_claims_method "P=[1000, 1100, 1200]" elr=0.7 "paid=[650,
500, 300]"
ultimate = [700.0,770.0,840.0]
reserve = [50.0,270.0,540.0]
total_reserve = 860
  by: Expected claims reserving method  (ultimate_i = P_i x ELR; reserve_i = ultimate_i - paid_i)
  elr = 0.7
  assumes: one paid amount per premium; 0 < ELR <= 500%
```

In the worked example, P = [1000, 1100, 1200], elr = 0.7, paid = [650, 500, 300] go through the equation above and give ultimate = [700, 770, 840], reserve = [50, 270, 540], total_reserve = 860. It applies only when one paid amount per premium; 0 < ELR <= 500%: outside that it says what is wrong instead of returning a number.

Mack's standard error of chain-ladder reserves — `act_mack_standard_error`

The standard errors (root mean squared errors of prediction) of the chain-ladder reserve of each origin year and of the total reserve by Mack's distribution-free model, with the variance parameters sigma^2 of each

development age.

$$\widehat{\operatorname{mse}}(\hat{R}_i) = \hat{C}_{iN}^2 \sum_{k=N-i}^{N-1} \frac{\hat{\sigma}_k^2}{\hat{f}_k^2} \left(\frac{1}{\hat{C}_{ik}} + \frac{1}{\sum_j C_{jk}} \right)$$

symbol		meaning	SI unit
`T`	in	cumulative loss triangle: origin years in rows, development ages in columns, unknown cells written as 0	—
`se`	out	standard error of each origin year's reserve (a list)	—
`total_se`	out	standard error of the total reserve	—
`sigma2`	out	Mack variance parameters sigma^2 of each age (a list)	—

Applies when: square triangle with N >= 4; known cells positive, unknown cells 0; last sigma^2 by Mack's extrapolation.

```
$ actuarial-science solve act_mack_standard_error "T=[[1000, 1800, 2100, 2200], [1100, 2000, 2300,
0], [1200, 2100, 0, 0], [1300, 0, 0, 0]]"
se = [0.0,15.2267485994,34.5563923039,72.2173151114]
total_se = 90.68083258
sigma2 = [1.43939393939,0.263157894737,0.0481119696749]
  by: Mack's standard error of chain-ladder reserves  (sigma_k^2 = sum C_ik (C_(i,k+1)/C_ik -
f_k)^2/(n_k - 1); mse(R_i) = C_iN^2 sum_k sigma_k^2/f_k^2 (1/C_ik + 1/S_k); total adds the
covariance terms)
  assumes: square triangle with N >= 4; known cells positive, unknown cells 0; last sigma^2 by
Mack's extrapolation
```

In the worked example, T = [1000 1800 2100 2200; 1100 2000 2300 0; 1200 2100 0 0; 1300 0 0 0] go through the equation above and give se = [0, 15.2267486, 34.5563923, 72.21731511], total_se = 90.68083258, sigma2 = [1.439393939, 0.2631578947, 0.04811196967]. It applies only when square triangle with N >= 4; known cells positive, unknown cells 0; last sigma^2 by Mack's extrapolation: outside that it says what is wrong instead of returning a number.

Multiple decrements (2)

Associated single decrement rates from a multiple decrement table (UDD) — `act_multiple_decrement_to_single`

The associated single decrement probabilities q'(1) and q'(2) from the two decrement probabilities of a double decrement table, assuming each decrement is uniformly distributed within the year in the multiple decrement table.

$$q_x'^{(j)} = 1 - \left(p_x^{(\tau)} \right)^{q_x^{(j)} / q_x^{(\tau)}}$$

symbol		meaning	SI unit
`q1`	in	probability of leaving by decrement 1 within the year q(1)	—
`q2`	in	probability of leaving by decrement 2 within the year q(2)	—

symbol		meaning	SI unit
`q1_single`	out	associated single decrement probability q'(1)	—
`q2_single`	out	associated single decrement probability q'(2)	—
`q_total`	out	total decrement probability q(tau)	—

Applies when: q1, q2 > 0; q1 + q2 < 1; UDD in the multiple decrement table.

```
$ actuarial-science solve act_multiple_decrement_to_single q1=0.1 q2=0.2
q1_single = 0.1120959983
q2_single = 0.2116264837
q_total = 0.3
  by: Associated single decrement rates from a multiple decrement table (UDD)  (q'(j) = 1 -
(p(tau))^(q(j)/q(tau)))
  q1 = 0.1
  q2 = 0.2
  assumes: q1, q2 > 0; q1 + q2 < 1; UDD in the multiple decrement table
```

In the worked example, q1 = 0.1, q2 = 0.2 go through the equation above and give q1_single = 0.1120959983, q2_single = 0.2116264837, q_total = 0.3. It applies only when q1, q2 > 0; q1 + q2 < 1; UDD in the multiple decrement table: outside that it says what is wrong instead of returning a number.

Multiple decrement probabilities from single decrement rates (UDD) — `act_single_to_multiple_decrement`

The double decrement probabilities q(1) and q(2) from the associated single decrement probabilities q'(1) and q'(2), each uniformly distributed within the year in its single decrement table.

$$q_x^{(1)} = q_x'^{(1)}\left(1 - \tfrac{1}{2}q_x'^{(2)}\right), \quad q_x^{(2)} = q_x'^{(2)}\left(1 - \tfrac{1}{2}q_x'^{(1)}\right)$$

symbol		meaning	SI unit
`q1s`	in	associated single decrement probability q'(1)	—
`q2s`	in	associated single decrement probability q'(2)	—
`q1`	out	probability of leaving by decrement 1, q(1)	—
`q2`	out	probability of leaving by decrement 2, q(2)	—
`q_total`	out	total decrement probability q(tau)	—

Applies when: 0 < q'(1), q'(2) < 1; UDD in each single decrement table.

```
$ actuarial-science solve act_single_to_multiple_decrement q1s=0.1 q2s=0.2
q1 = 0.09
q2 = 0.19
q_total = 0.28
  by: Multiple decrement probabilities from single decrement rates (UDD)  (q(1) = q'(1) (1 -
q'(2)/2), q(2) = q'(2) (1 - q'(1)/2))
  q1s = 0.1
  q2s = 0.2
  assumes: 0 < q'(1), q'(2) < 1; UDD in each single decrement table
```

In the worked example, q1s = 0.1, q2s = 0.2 go through the equation above and give q1 = 0.09, q2 = 0.19, q_total = 0.28. It applies only when 0 < q'(1), q'(2) < 1; UDD in each single decrement table: outside that it says what is wrong instead of returning a number.

Multiple lives (2)

Joint life and last survivor annuities and insurances — `act_joint_life_last_survivor`

The joint life and last survivor annuities-due and whole life insurances (end of year of the first or second death) for two independent lives aged x and y with their own life tables.

$$\ddot{a}_{xy} = \sum_{k \geq 0} v^k \, {}_kp_x \, {}_kp_y, \quad \ddot{a}_{\overline{xy}} = \ddot{a}_x + \ddot{a}_y - \ddot{a}_{xy}, \quad A = 1 - d\,\ddot{a}$$

symbol		meaning	SI unit
`qx`	in	death probabilities of the first life from age x, ending with 1 (a list)	—
`qy`	in	death probabilities of the second life from age y, ending with 1 (a list)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`a_joint`	out	joint life annuity-due ä_xy	—
`a_last`	out	last survivor annuity-due	—
`A_joint`	out	joint life insurance A_xy	—
`A_last`	out	last survivor insurance	—

Applies when: both tables close; independent lives; 0 < i < 1.

```
$ actuarial-science solve act_joint_life_last_survivor "qx=[0.1, 0.2, 0.5, 1]" "qy=[0.05, 0.1, 0.3,
0.6, 1]" i=0.05
a_joint = 2.55877551
a_last = 3.656643991
A_joint = 0.8781535471
A_last = 0.8258740957
  by: Joint life and last survivor annuities and insurances  (ä_xy = sum v^k k_p_x k_p_y; last
survivor ä = ä_x + ä_y - ä_xy; A = 1 - d ä)
  i = 0.05
  assumes: both tables close; independent lives; 0 < i < 1
```

In the worked example, qx = [0.1, 0.2, 0.5, 1], qy = [0.05, 0.1, 0.3, 0.6, 1], i = 0.05 go through the equation above and give a_joint = 2.55877551, a_last = 3.656643991, A_joint = 0.8781535471, A_last = 0.8258740957. It applies only when both tables close; independent lives; 0 < i < 1: outside that it says what is wrong instead of returning a number.

Reversionary annuity — `act_reversionary_annuity`

The actuarial present value of an annuity-immediate of P a year paid to a life aged y after the death of a life aged x (a_(x|y) = a_y - a_xy), independent lives.

$$a_{x|y} = a_y - a_{xy} = \sum_{k \geq 1} v^k \, {}_kp_y \, (1 - {}_kp_x)$$

symbol		meaning	SI unit
`qx`	in	death probabilities of the life whose death starts the annuity, ending with 1 (a list)	—
`qy`	in	death probabilities of the annuitant from age y, ending with 1 (a list)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`P`	in	annual payment	—
`a_rev`	out	reversionary annuity a_(x|y) per unit	—
`EPV`	out	actuarial present value P a_(x|y)	—

Applies when: both tables close; independent lives; 0 < i < 1.

```
$ actuarial-science solve act_reversionary_annuity "qx=[0.1, 0.2, 0.5, 1]" "qy=[0.05, 0.1, 0.3,
0.6, 1]" i=0.05 P=1000
a_rev = 0.8354583738
EPV = 835.4583738
  by: Reversionary annuity  (a_(x|y) = a_y - a_xy = sum_(k>=1) v^k k_p_y (1 - k_p_x))
  i = 0.05
  P = 1000
  assumes: both tables close; independent lives; 0 < i < 1
```

In the worked example, qx = [0.1, 0.2, 0.5, 1], qy = [0.05, 0.1, 0.3, 0.6, 1], i = 0.05, P = 1000 go through the equation above and give a_rev = 0.8354583738, EPV = 835.4583738. It applies only when both tables close; independent lives; 0 < i < 1: outside that it says what is wrong instead of returning a number.

Premiums (6)

Gross annual premium for whole life insurance with expenses — `act_gross_premium`

The level annual gross premium for a fully discrete whole life insurance of S by the equivalence principle with expenses: an initial fraction e0 of the first premium, a renewal fraction r of later premiums, a per-policy expense c every year (first year included) and a settlement expense E per claim; with the net premium and the loading.

$$G = \frac{(S+E)A_x + c\,\ddot{a}_x}{(1-r)\ddot{a}_x - (e_0 - r)}$$

symbol		meaning	SI unit
`q`	in	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`S`	in	sum insured	—
`e0`	in	initial expense as a fraction of the first gross premium (e.g. 40%)	—
`r`	in	renewal expense as a fraction of each later gross premium (e.g. 5%)	—
`c`	in	per-policy expense at the start of every year	—
`E`	in	settlement expense per claim	—
`G`	out	annual gross premium	—
`P`	out	annual net premium	—

symbol		meaning	SI unit
`loading`	out	expense loading G - P	—

Applies when: the table closes; 0 < i < 1; 0 <= e0, r < 1; c, E >= 0.

```
$ actuarial-science solve act_gross_premium "q=[0.1, 0.2, 0.5, 1]" i=0.05 S=1000 e0=0.4 r=0.05 c=10
E=20
G = 391.0443937
P = 306.8418549
loading = 84.20253884
  by: Gross annual premium for whole life insurance with expenses  (G ä_x = (S + E) A_x + e0 G + r
G (ä_x - 1) + c ä_x)
  i = 0.05
  S = 1000
  e0 = 0.4
  r = 0.05
  c = 10
  E = 20
  assumes: the table closes; 0 < i < 1; 0 <= e0, r < 1; c, E >= 0
```

In the worked example, q = [0.1, 0.2, 0.5, 1], i = 0.05, S = 1000, e0 = 0.4, r = 0.05, c = 10, E = 20 go through the equation above and give G = 391.0443937, P = 306.8418549, loading = 84.20253884. It applies only when the table closes; 0 < i < 1; 0 <= e0, r < 1; c, E >= 0: outside that it says what is wrong instead of returning a number.

Loss at issue of whole life insurance: mean, standard deviation, probability of loss — `act_loss_at_issue`

The mean and standard deviation of the insurer's loss at issue L0 = S v^(K+1) - P ä_(K+1) for a fully discrete whole life insurance of S with annual premium P, and the probability that the loss is positive.

$$L_0 = \left(S + \frac{P}{d}\right) v^{K+1} - \frac{P}{d}, \quad \mathrm{Var}(L_0) = \left(S + \frac{P}{d}\right)^2 \left({}^2A_x - A_x^2\right)$$

symbol		meaning	SI unit
`q`	in	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`S`	in	sum insured	—
`P`	in	annual premium	—
`mean`	out	expected loss at issue S A_x - P ä_x	—
`sd`	out	standard deviation (S + P/d) sqrt(2A_x - A_x^2)	—
`prob_loss`	out	probability that the loss at issue is positive	—

Applies when: the table closes; 0 < i < 1; S, P > 0.

```
$ actuarial-science solve act_loss_at_issue "q=[0.1, 0.2, 0.5, 1]" i=0.05 S=1000 P=450
mean = -403.8756999
sd = 434.1005558
prob_loss = 0.28
  by: Loss at issue of whole life insurance: mean, standard deviation, probability of loss  (L0 =
(S + P/d) v^(K+1) - P/d; E = S A - P ä; Var = (S + P/d)^2 (2A - A^2))
  i = 0.05
  S = 1000
  P = 450
  assumes: the table closes; 0 < i < 1; S, P > 0
```

In the worked example, q = [0.1, 0.2, 0.5, 1], i = 0.05, S = 1000, P = 450 go through the equation above and give mean = -403.8756999, sd = 434.1005558, prob_loss = 0.28. It applies only when the table closes; 0 < i < 1; S, P > 0: outside that it says what is wrong instead of returning a number.

Net annual premium for endowment insurance — `act_net_premium_endowment`

The level annual net premium, payable for n years while alive, for a fully discrete n-year endowment insurance of S, by the equivalence principle.

$$P = S\,\frac{A_{x:\overline{n}|}}{\ddot{a}_{x:\overline{n}|}} = S\left(\frac{1}{\ddot{a}_{x:\overline{n}|}} - d\right)$$

symbol		meaning	SI unit
`q`	in	death probabilities q_x, q_(x+1), ... for at least n years (a list)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`n`	in	term in years	—
`S`	in	sum insured	—
`P`	out	annual net premium S A_(x:n) / ä_(x:n)	—
`A`	out	endowment insurance A_(x:n)	—
`a_due`	out	temporary annuity-due ä_(x:n)	—

Applies when: at least n death probabilities; 0 < i < 1; S > 0.

```
$ actuarial-science solve act_net_premium_endowment "q=[0.1, 0.2, 0.5, 1]" i=0.05 n=3 S=1000
P = 350.7549361
A = 0.8804664723
a_due = 2.510204082
  by: Net annual premium for endowment insurance  (P ä_(x:n) = S A_(x:n); equivalently P = S (1/
ä_(x:n) - d))
  i = 0.05
  S = 1000
  assumes: at least n death probabilities; 0 < i < 1; S > 0
```

In the worked example, q = [0.1, 0.2, 0.5, 1], i = 0.05, n = 3, S = 1000 go through the equation above and give P = 350.7549361, A = 0.8804664723, a_due = 2.510204082. It applies only when at least n death probabilities; 0 < i < 1; S > 0: outside that it says what is wrong instead of returning a number.

Net annual premium for limited-payment whole life insurance — `act_net_premium_limited_pay`

The level annual net premium for a fully discrete whole life insurance of S with premiums payable for at most h years.

$$ {}_hP_x = S\,\frac{A_x}{\ddot{a}_{x:\overline{h}|}} $$

symbol		meaning	SI unit
`q`	in	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`h`	in	premium paying period in years	—
`S`	in	sum insured	—
`P`	out	annual net premium S A_x / ä_(x:h)	—
`A`	out	whole life insurance A_x	—
`a_due_h`	out	h-year temporary annuity-due	—

Applies when: the table closes; 1 <= h <= length of the table; 0 < i < 1.

```
$ actuarial-science solve act_net_premium_limited_pay "q=[0.1, 0.2, 0.5, 1]" i=0.05 h=2 S=1000
P = 466.1234457
A = 0.8656578278
a_due_h = 1.857142857
  by: Net annual premium for limited-payment whole life insurance  (P ä_(x:h) = S A_x)
  i = 0.05
  S = 1000
  assumes: the table closes; 1 <= h <= length of the table; 0 < i < 1
```

In the worked example, q = [0.1, 0.2, 0.5, 1], i = 0.05, h = 2, S = 1000 go through the equation above and give P = 466.1234457, A = 0.8656578278, a_due_h = 1.857142857. It applies only when the table closes; 1 <= h <= length of the table; 0 < i < 1: outside that it says what is wrong instead of returning a number.

Net annual premium for term insurance — `act_net_premium_term`

The level annual net premium, payable for n years while alive, for a fully discrete n-year term insurance of S, by the equivalence principle.

$$ P = S\,\frac{A^{1}_{x:\overline{n}|}}{\ddot{a}_{x:\overline{n}|}} $$

symbol		meaning	SI unit
`q`	in	death probabilities q_x, q_(x+1), ... for at least n years (a list)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`n`	in	term in years	—
`S`	in	sum insured	—
`P`	out	annual net premium S A^1_(x:n) / ä_(x:n)	—
`A1`	out	term insurance A^1_(x:n)	—

symbol		meaning	SI unit
`a_due`	out	temporary annuity-due ä_(x:n)	—

Applies when: at least n death probabilities; 0 < i < 1; S > 0.

```
$ actuarial-science solve act_net_premium_term "q=[0.1, 0.2, 0.5, 1]" i=0.05 n=3 S=1000
P = 226.867983
A1 = 0.5694849368
a_due = 2.510204082
  by: Net annual premium for term insurance  (P ä_(x:n) = S A^1_(x:n))
  i = 0.05
  S = 1000
  assumes: at least n death probabilities; 0 < i < 1; S > 0
```

In the worked example, q = [0.1, 0.2, 0.5, 1], i = 0.05, n = 3, S = 1000 go through the equation above and give P = 226.867983, A1 = 0.5694849368, a_due = 2.510204082. It applies only when at least n death probabilities; 0 < i < 1; S > 0: outside that it says what is wrong instead of returning a number.

Net annual premium for whole life insurance — `act_net_premium_whole_life`

The level annual net premium, payable at the start of each year while alive, for a fully discrete whole life insurance of S, by the equivalence principle.

$$P = S\,\frac{A_x}{\ddot{a}_x} = S\left(\frac{1}{\ddot{a}_x} - d\right)$$

symbol		meaning	SI unit
`q`	in	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`S`	in	sum insured	—
`P`	out	annual net premium S A_x / ä_x	—
`A`	out	whole life insurance A_x	—
`a_due`	out	whole life annuity-due ä_x	—

Applies when: the table closes; 0 < i < 1; S > 0.

```
$ actuarial-science solve act_net_premium_whole_life "q=[0.1, 0.2, 0.5, 1]" i=0.05 S=1000
P = 306.8418549
A = 0.8656578278
a_due = 2.821185617
  by: Net annual premium for whole life insurance  (equivalence principle: P ä_x = S A_x)
  i = 0.05
  S = 1000
  assumes: the table closes; 0 < i < 1; S > 0
```

In the worked example, q = [0.1, 0.2, 0.5, 1], i = 0.05, S = 1000 go through the equation above and give P = 306.8418549, A = 0.8656578278, a_due = 2.821185617. It applies only when the table closes; 0 < i < 1; S > 0: outside that it says what is wrong instead of returning a number.

Ratemaking (2)

Indicated rate by the pure premium method — `act_pure_premium_rate`

The pure premium (losses per exposure) and the indicated rate per exposure from projected losses, exposures, the fixed expense per exposure, the variable expense ratio and the profit provision.

$$R = \frac{L/X + F}{1 - V - Q}$$

symbol		meaning	SI unit
`L`	in	projected ultimate losses and LAE	—
`X`	in	earned exposures	—
`Fe`	in	fixed expense per exposure	—
`V`	in	variable expense ratio	—
`Q`	in	profit and contingencies provision	—
`pure_premium`	out	pure premium L/X	—
`rate`	out	indicated rate per exposure (PP + F)/(1 - V - Q)	—

Applies when: X > 0; 0 <= V < 1; V + Q < 1.

```
$ actuarial-science solve act_pure_premium_rate L=500000 X=2000 Fe=30 V=0.15 Q=0.05
pure_premium = 250
rate = 350
  by: Indicated rate by the pure premium method  (rate = (L/X + F)/(1 - V - Q))
  L = 500000
  X = 2000
  Fe = 30
  V = 0.15
  Q = 0.05
  assumes: X > 0; 0 <= V < 1; V + Q < 1
```

In the worked example, L = 500000, X = 2000, Fe = 30, V = 0.15, Q = 0.05 go through the equation above and give pure_premium = 250, rate = 350. It applies only when X > 0; 0 <= V < 1; V + Q < 1: outside that it says what is wrong instead of returning a number.

Indicated rate change by the loss ratio method — `act_loss_ratio_rate_change`

The indicated rate change from projected losses (with loss adjustment expenses), earned premium at current rates, the fixed expense ratio, the variable expense ratio and the profit provision.

$$\Delta = \frac{L/EP + F}{1 - V - Q} - 1$$

symbol		meaning	SI unit
`L`	in	projected ultimate losses and LAE	—
`EP`	in	earned premium at current rates	—

symbol		meaning	SI unit
`F`	in	fixed expense ratio (to premium)	—
`V`	in	variable expense ratio	—
`Q`	in	profit and contingencies provision	—
`change`	out	indicated rate change (0.05 = +5%)	—
`loss_ratio`	out	projected loss ratio L/EP	—
`permissible`	out	permissible loss ratio 1 - V - Q	—

Applies when: EP > 0; 0 <= F, V < 1; V + Q < 1.

```
$ actuarial-science solve act_loss_ratio_rate_change L=650000 EP=1000000 F=0.1 V=0.15 Q=0.05
change = -0.0625
loss_ratio = 0.65
permissible = 0.8
  by: Indicated rate change by the loss ratio method  (indicated change = (L/EP + F)/(1 - V - Q) -
1)
  L = 650000
  EP = 1000000
  F = 0.1
  V = 0.15
  Q = 0.05
  assumes: EP > 0; 0 <= F, V < 1; V + Q < 1
```

In the worked example, L = 650000, EP = 1000000, F = 0.1, V = 0.15, Q = 0.05 go through the equation above and give change = -0.0625, loss_ratio = 0.65, permissible = 0.8. It applies only when EP > 0; 0 <= F, V < 1; V + Q < 1: outside that it says what is wrong instead of returning a number.

Reserves (8)

Continuous whole life insurance, premium and reserve under Makeham's law — `act_makeham_continuous_whole_life`

The continuous whole life annuity, the insurance paid at the moment of death, the continuous net premium rate and the reserve at time t for a life aged x following Makeham's law, at a constant interest rate.

$$\bar{a}_x = \int_0^\infty e^{-\delta s}\, {}_sp_x \, ds, \quad \bar{A}_x = 1 - \delta \bar{a}_x, \quad {}_t\bar{V} = S\left(1 - \frac{\bar{a}_{x+t}}{\bar{a}_x}\right)$$

symbol		meaning	SI unit
`A`	in	Makeham's A (per year)	—
`B`	in	Makeham's B (per year)	—
`c`	in	Makeham's c (> 1)	—
`x`	in	age at issue (years)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`S`	in	sum insured	—
`t`	in	time since issue in years	—
`a_bar`	out	continuous whole life annuity ā_x	—

symbol		meaning	SI unit
`A_bar`	out	insurance payable at death Ā_x	—
`P`	out	continuous annual net premium rate	—
`V`	out	reserve at time t	—

Applies when: Makeham parameters 0 <= A < 0.1, 0 < B < 0.1, 1 < c < 1.5; 0 <= x <= 120; 0 <= t <= 100; 0 < i < 1.

```
$ actuarial-science solve act_makeham_continuous_whole_life A=0.00022 B=2.7e-06 c=1.124 x=50 i=0.05
S=100000 t=10
a_bar = 16.52037321
A_bar = 0.1939682791
P = 1174.1156
V = 12836.47174
  by: Continuous whole life insurance, premium and reserve under Makeham's law  (ā_x = integral of
e^(-delta s) s_p_x (Gauss-Legendre); Ā_x = 1 - delta ā_x; P = S Ā_x/ā_x; tV = S (1 - ā_(x+t)/ā_x))
  A = 0.00022
  B = 2.7e-06
  c = 1.124
  x = 50
  i = 0.05
  S = 100000
  t = 10
  assumes: Makeham parameters 0 <= A < 0.1, 0 < B < 0.1, 1 < c < 1.5; 0 <= x <= 120; 0 <= t <= 100;
0 < i < 1
```

In the worked example, A = 0.00022, B = 2.7e-06, c = 1.124, x = 50, i = 0.05, S = 100000, t = 10 go through the equation above and give a_bar = 16.52037321, A_bar = 0.1939682791, P = 1174.1156, V = 12836.47174. It applies only when Makeham parameters 0 <= A < 0.1, 0 < B < 0.1, 1 < c < 1.5; 0 <= x <= 120; 0 <= t <= 100; 0 < i < 1: outside that it says what is wrong instead of returning a number.

Gross premium reserve for whole life insurance with expenses — `act_gross_premium_reserve`

The gross premium policy value at integer duration t of a fully discrete whole life insurance of S, with the gross premium set by the equivalence principle with initial, renewal, per-policy and settlement expenses.

$$ {}_tV^g = (S+E)A_{x+t} + c\,\ddot{a}_{x+t} - (1-r)G\,\ddot{a}_{x+t} $$

symbol		meaning	SI unit
`q`	in	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`S`	in	sum insured	—
`e0`	in	initial expense as a fraction of the first gross premium	—
`r`	in	renewal expense as a fraction of each later gross premium	—
`c`	in	per-policy expense at the start of every year	—
`E`	in	settlement expense per claim	—
`t`	in	policy duration in years	—
`V`	out	gross premium reserve tV	—

symbol		meaning	SI unit
`G`	out	annual gross premium	—

Applies when: the table closes; 0 <= t < length of the table; 0 <= e0, r < 1.

```
$ actuarial-science solve act_gross_premium_reserve "q=[0.1, 0.2, 0.5, 1]" i=0.05 S=1000 e0=0.4
r=0.05 c=10 E=20 t=1
V = 148.7310756
G = 391.0443937
  by: Gross premium reserve for whole life insurance with expenses  (tV = (S + E) A_(x+t) + c
ä_(x+t) - G (1 - r) ä_(x+t) for t >= 1 (0 at issue by the equivalence principle))
  i = 0.05
  S = 1000
  e0 = 0.4
  r = 0.05
  c = 10
  E = 20
  assumes: the table closes; 0 <= t < length of the table; 0 <= e0, r < 1
```

In the worked example, q = [0.1, 0.2, 0.5, 1], i = 0.05, S = 1000, e0 = 0.4, r = 0.05, c = 10, E = 20, t = 1 go through the equation above and give V = 148.7310756, G = 391.0443937. It applies only when the table closes; 0 <= t < length of the table; 0 <= e0, r < 1: outside that it says what is wrong instead of returning a number.

Net premium reserve for endowment insurance — `act_reserve_endowment`

The net premium reserve at integer duration t of a fully discrete n-year endowment insurance of S with level net premiums payable for n years, by the prospective method (S at maturity).

$$ {}_tV = S\,A_{x+t:\overline{n-t}|} - P\,\ddot{a}_{x+t:\overline{n-t}|} $$

symbol		meaning	SI unit
`q`	in	death probabilities q_x, q_(x+1), ... for at least n years (a list)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`n`	in	term in years	—
`S`	in	sum insured	—
`t`	in	policy duration in years	—
`V`	out	net premium reserve tV	—
`P`	out	annual net premium	—

Applies when: at least n death probabilities; 0 <= t <= n; 0 < i < 1.

```
$ actuarial-science solve act_reserve_endowment "q=[0.1, 0.2, 0.5, 1]" i=0.05 n=3 S=1000 t=1
V = 298.102981
P = 350.7549361
  by: Net premium reserve for endowment insurance  (tV = S A_(x+t:n-t) - P ä_(x+t:n-t))
  i = 0.05
  S = 1000
  assumes: at least n death probabilities; 0 <= t <= n; 0 < i < 1
```

In the worked example, q = [0.1, 0.2, 0.5, 1], i = 0.05, n = 3, S = 1000, t = 1 go through the equation above and give V = 298.102981, P = 350.7549361. It applies only when at least n death probabilities; 0 <= t <= n; 0 < i < 1: outside that it says what is wrong instead of returning a number.

Net premium reserve for term insurance — `act_reserve_term`

The net premium reserve at integer duration t of a fully discrete n-year term insurance of S with level net premiums payable for n years, by the prospective method.

$$ {}_tV = S\,A^{1}_{x+t:\overline{n-t}|} - P\,\ddot{a}_{x+t:\overline{n-t}|} $$

symbol		meaning	SI unit
`q`	in	death probabilities q_x, q_(x+1), ... for at least n years (a list)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`n`	in	term in years	—
`S`	in	sum insured	—
`t`	in	policy duration in years	—
`V`	out	net premium reserve tV	—
`P`	out	annual net premium	—

Applies when: at least n death probabilities; 0 <= t <= n; 0 < i < 1.

```
$ actuarial-science solve act_reserve_term "q=[0.01, 0.02, 0.04, 0.08]" i=0.05 n=4 S=1000 t=2
V = 43.21035282
P = 33.71740631
  by: Net premium reserve for term insurance  (tV = S A^1_(x+t:n-t) - P ä_(x+t:n-t))
  i = 0.05
  S = 1000
  assumes: at least n death probabilities; 0 <= t <= n; 0 < i < 1
```

In the worked example, q = [0.01, 0.02, 0.04, 0.08], i = 0.05, n = 4, S = 1000, t = 2 go through the equation above and give V = 43.21035282, P = 33.71740631. It applies only when at least n death probabilities; 0 <= t <= n; 0 < i < 1: outside that it says what is wrong instead of returning a number.

Net premium reserve for whole life insurance — `act_reserve_whole_life`

The net premium reserve (policy value) at integer duration t of a fully discrete whole life insurance of S with the equivalence-principle net premium, by the prospective method.

$$ {}_tV = S\,A_{x+t} - P\,\ddot{a}_{x+t} = S\left(1 - \frac{\ddot{a}_{x+t}}{\ddot{a}_x}\right) $$

symbol		meaning	SI unit
`q`	in	death probabilities q_x, q_(x+1), ... ending with 1 (a list)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`S`	in	sum insured	—

symbol		meaning	SI unit
`t`	in	policy duration in years	—
`V`	out	net premium reserve tV	—
`P`	out	annual net premium	—
`A_t`	out	A_(x+t)	—
`a_due_t`	out	ä_(x+t)	—

Applies when: the table closes; 0 <= t < length of the table; 0 < i < 1.

```
$ actuarial-science solve act_reserve_whole_life "q=[0.1, 0.2, 0.5, 1]" i=0.05 S=1000 t=1
V = 246.8710529
P = 306.8418549
A_t = 0.8988230213
a_due_t = 2.124716553
  by: Net premium reserve for whole life insurance  (tV = S A_(x+t) - P ä_(x+t) = S (1 - ä_(x+t)/
ä_x))
  i = 0.05
  S = 1000
  assumes: the table closes; 0 <= t < length of the table; 0 < i < 1
```

In the worked example, q = [0.1, 0.2, 0.5, 1], i = 0.05, S = 1000, t = 1 go through the equation above and give V = 246.8710529, P = 306.8418549, A_t = 0.8988230213, a_due_t = 2.124716553. It applies only when the table closes; 0 <= t < length of the table; 0 < i < 1: outside that it says what is wrong instead of returning a number.

Policy value recursion (one year) — `act_policy_value_recursion`

The policy value at the end of a policy year from the value at its start, the premium and expense at the start of the year, the interest rate, the death probability, the sum insured and the settlement expense: (tV + P - e)(1+i) = q (S + E) + p (t+1)V.

$$({}_tV + P - e)(1 + i) = q_{x+t}\,(S + E) + p_{x+t}\;{}_{t+1}V$$

symbol		meaning	SI unit
`V`	in	policy value at the start of the year tV	—
`P`	in	premium at the start of the year	—
`e`	in	expense at the start of the year	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`q`	in	death probability for the year q_(x+t)	—
`S`	in	death benefit	—
`E`	in	settlement expense per claim	—
`V_next`	out	policy value at the end of the year (t+1)V	—

Applies when: 0 <= q < 1; 0 < i < 1.

```
$ actuarial-science solve act_policy_value_recursion V=100 P=50 e=5 i=0.05 q=0.01 S=1000 E=0
V_next = 143.6868687
  by: Policy value recursion (one year)  ((t+1)V = [(tV + P - e)(1 + i) - q (S + E)] / (1 - q))
  V = 100
  P = 50
  e = 5
  i = 0.05
  q = 0.01
  S = 1000
  E = 0
  assumes: 0 <= q < 1; 0 < i < 1
```

In the worked example, V = 100, P = 50, e = 5, i = 0.05, q = 0.01, S = 1000, E = 0 go through the equation above and give V_next = 143.6868687. It applies only when 0 <= q < 1; 0 < i < 1: outside that it says what is wrong instead of returning a number.

Retrospective reserve (accumulated fund per survivor) — `act_retrospective_reserve`

The retrospective reserve at integer duration t of a fully discrete life insurance of S with annual premium P: the actuarial accumulated value of the premiums received less the cost of insurance, per survivor.

$$ {}_tV = \frac{P\,\ddot{a}_{x:\overline{t}|} - S\,A^{1}_{x:\overline{t}|}}{{}_tE_x} $$

symbol		meaning	SI unit
`q`	in	death probabilities q_x, q_(x+1), ... for at least t years (a list)	—
`i`	in	annual effective interest rate (5% or 0.05)	—
`S`	in	sum insured	—
`P`	in	annual premium	—
`t`	in	policy duration in years	—
`V`	out	retrospective reserve tV	—
`tEx`	out	pure endowment t_E_x	—

Applies when: at least t death probabilities; t_p_x > 0; 0 < i < 1.

```
$ actuarial-science solve act_retrospective_reserve "q=[0.1, 0.2, 0.5, 1]" i=0.05 S=1000 P=450 t=2
V = 883.8541667
tEx = 0.6530612245
  by: Retrospective reserve (accumulated fund per survivor)  (tV = (P ä_(x:t) - S A^1_(x:t)) /
t_E_x)
  i = 0.05
  S = 1000
  P = 450
  assumes: at least t death probabilities; t_p_x > 0; 0 < i < 1
```

In the worked example, q = [0.1, 0.2, 0.5, 1], i = 0.05, S = 1000, P = 450, t = 2 go through the equation above and give V = 883.8541667, tEx = 0.6530612245. It applies only when at least t death probabilities; t_p_x > 0; 0 < i < 1: outside that it says what is wrong instead of returning a number.

Thiele's equation: continuous endowment insurance reserve (constant forces) — `act_thiele_endowment`

The continuous net premium rate and the reserve at time t of an n-year endowment insurance of S paid at the moment of death or at maturity, with a constant force of mortality mu and a constant force of interest delta (the solution of Thiele's equation dV/dt = delta V + P - mu (S - V), V(n) = S).

$$\frac{d\,{}_tV}{dt} = \delta\,{}_tV + \bar{P} - \mu\,(S - {}_tV), \quad {}_nV = S$$

symbol		meaning	SI unit
`mu`	in	constant force of mortality per year	—
`delta`	in	constant force of interest per year	—
`n`	in	term in years	—
`S`	in	sum insured	—
`t`	in	time since issue in years	—
`P`	out	continuous annual net premium rate	—
`A_bar`	out	endowment insurance Ā_(x:n)	—
`a_bar`	out	continuous temporary annuity ā_(x:n)	—
`V`	out	reserve at time t	—

Applies when: 0 < mu, delta < 1; 0 < n <= 100; 0 <= t <= n.

```
$ actuarial-science solve act_thiele_endowment mu=0.02 delta=0.05 n=20 S=1000 t=10
P = 42.91175725
A_bar = 0.4618549742
a_bar = 10.76290052
V = 331.8122278
  by: Thiele's equation: continuous endowment insurance reserve (constant forces)  (Ā =
mu/(mu+delta)(1 - e^(-(mu+delta)n)) + e^(-(mu+delta)n); ā = (1 - e^(-(mu+delta)n))/(mu+delta); P =
S Ā/ā; tV = S Ā_(n-t) - P ā_(n-t))
  mu = 0.02
  delta = 0.05
  n = 20
  S = 1000
  t = 10
  assumes: 0 < mu, delta < 1; 0 < n <= 100; 0 <= t <= n
```

In the worked example, mu = 0.02, delta = 0.05, n = 20, S = 1000, t = 10 go through the equation above and give P = 42.91175725, A_bar = 0.4618549742, a_bar = 10.76290052, V = 331.8122278. It applies only when 0 < mu, delta < 1; 0 < n <= 100; 0 <= t <= n: outside that it says what is wrong instead of returning a number.

Risk measures (4)

VaR and TVaR of a lognormal loss — `act_var_tvar_lognormal`

The value at risk and tail value at risk at security level p of a lognormal loss (mu, sigma of the log of the loss).

$$\mathrm{VaR}_p = e^{\mu+\sigma z_p}, \quad \mathrm{TVaR}_p = e^{\mu+\sigma^2/2}\,\frac{\Phi(\sigma - z_p)}{1-p}$$

symbol		meaning	SI unit
`mu`	in	mean of the log of the loss	—
`sigma`	in	standard deviation of the log of the loss	—
`p`	in	security level (95% or 0.95)	—
`VaR`	out	value at risk at level p (the p-quantile)	—
`TVaR`	out	tail value at risk E[X | X > VaR]	—

Applies when: sigma > 0; 0 < p < 1.

```
$ actuarial-science solve act_var_tvar_lognormal mu=7 sigma=1 p=0.95
VaR = 5680.835676
TVaR = 9384.138726
  by: VaR and TVaR of a lognormal loss  (VaR = exp(mu + sigma z_p); TVaR = e^(mu + sigma^2/2)
Phi(sigma - z_p)/(1 - p))
  mu = 7
  sigma = 1
  p = 0.95
  assumes: sigma > 0; 0 < p < 1
```

In the worked example, mu = 7, sigma = 1, p = 0.95 go through the equation above and give VaR = 5680.835676, TVaR = 9384.138726. It applies only when sigma > 0; 0 < p < 1: outside that it says what is wrong instead of returning a number.

VaR and TVaR of a normal loss — `act_var_tvar_normal`

The value at risk and tail value at risk at security level p of a normally distributed loss.

$$\mathrm{VaR}_p = \mu + \sigma z_p, \quad \mathrm{TVaR}_p = \mu + \sigma\frac{\phi(z_p)}{1-p}$$

symbol		meaning	SI unit
`mu`	in	mean loss	—
`sigma`	in	standard deviation of the loss	—
`p`	in	security level (95% or 0.95)	—
`VaR`	out	value at risk at level p (the p-quantile)	—
`TVaR`	out	tail value at risk E[X | X > VaR]	—

Applies when: sigma > 0; 0 < p < 1.

```
$ actuarial-science solve act_var_tvar_normal mu=1000 sigma=200 p=0.95
VaR = 1328.970725
TVaR = 1412.542561
  by: VaR and TVaR of a normal loss  (VaR = mu + sigma z_p; TVaR = mu + sigma phi(z_p)/(1 - p))
  mu = 1000
  sigma = 200
  p = 0.95
  assumes: sigma > 0; 0 < p < 1
```

In the worked example, mu = 1000, sigma = 200, p = 0.95 go through the equation above and give VaR = 1328.970725, TVaR = 1412.542561. It applies only when sigma > 0; 0 < p < 1: outside that it says what is wrong instead of returning a number.

VaR and TVaR of a Pareto loss — `act_var_tvar_pareto`

The value at risk and tail value at risk at security level p of a two-parameter Pareto loss (alpha, theta).

$$\mathrm{VaR}_p = \theta\left[(1-p)^{-1/\alpha} - 1\right], \quad \mathrm{TVaR}_p = \mathrm{VaR}_p + \frac{\theta + \mathrm{VaR}_p}{\alpha - 1}$$

symbol		meaning	SI unit
`alpha`	in	Pareto shape alpha (> 1)	—
`theta`	in	Pareto scale theta	—
`p`	in	security level (95% or 0.95)	—
`VaR`	out	value at risk at level p (the p-quantile)	—
`TVaR`	out	tail value at risk E[X | X > VaR]	—

Applies when: alpha > 1; theta > 0; 0 < p < 1.

```
$ actuarial-science solve act_var_tvar_pareto alpha=3 theta=2000 p=0.95
VaR = 3428.835233
TVaR = 6143.25285
  by: VaR and TVaR of a Pareto loss  (VaR = theta [(1-p)^(-1/alpha) - 1]; TVaR = VaR + (theta +
VaR)/(alpha - 1))
  alpha = 3
  theta = 2000
  p = 0.95
  assumes: alpha > 1; theta > 0; 0 < p < 1
```

In the worked example, alpha = 3, theta = 2000, p = 0.95 go through the equation above and give VaR = 3428.835233, TVaR = 6143.25285. It applies only when alpha > 1; theta > 0; 0 < p < 1: outside that it says what is wrong instead of returning a number.

VaR and TVaR of an exponential loss — `act_var_tvar_exponential`

The value at risk and tail value at risk at security level p of an exponentially distributed loss with mean theta.

$$\mathrm{VaR}_p = -\theta\ln(1-p), \quad \mathrm{TVaR}_p = \mathrm{VaR}_p + \theta$$

symbol		meaning	SI unit
`theta`	in	mean loss theta	—
`p`	in	security level (95% or 0.95)	—
`VaR`	out	value at risk at level p (the p-quantile)	—
`TVaR`	out	tail value at risk E[X | X > VaR]	—

Applies when: theta > 0; 0 < p < 1.

```
$ actuarial-science solve act_var_tvar_exponential theta=1000 p=0.95
VaR = 2995.732274
TVaR = 3995.732274
  by: VaR and TVaR of an exponential loss  (VaR = -theta ln(1 - p); TVaR = VaR + theta)
  theta = 1000
  p = 0.95
  assumes: theta > 0; 0 < p < 1
```

In the worked example, theta = 1000, p = 0.95 go through the equation above and give VaR = 2995.732274, TVaR = 3995.732274. It applies only when theta > 0; 0 < p < 1: outside that it says what is wrong instead of returning a number.

Survival estimation (3)

Exponential maximum-likelihood fit with censoring — `act_exponential_mle`

The maximum-likelihood constant hazard (deaths divided by total time observed), the mean lifetime and the fitted survival probability at time t from times with death (1) or censoring (0) indicators.

$$\hat{\lambda} = \frac{\sum_i \delta_i}{\sum_i t_i}, \quad \hat{S}(t) = e^{-\hat{\lambda} t}$$

symbol		meaning	SI unit
`times`	in	observed times (a list)	—
`events`	in	1 for a death or failure, 0 for censoring (a list)	—
`t`	in	time for the fitted survival probability	—
`hazard`	out	maximum-likelihood hazard rate (deaths / total exposure)	—
`mean`	out	fitted mean lifetime	—
`S`	out	fitted survival probability exp(-lambda t)	—

Applies when: one indicator per time; positive times; at least one death.

```
$ actuarial-science solve act_exponential_mle "times=[1, 2, 2, 3, 5]" "events=[1, 1, 0, 1, 0]" t=3
hazard = 0.2307692308
mean = 4.333333333
S = 0.5004199204
  by: Exponential maximum-likelihood fit with censoring  (lambda = sum of death indicators / sum of
times; S(t) = e^(-lambda t))
  t = 3
  assumes: one indicator per time; positive times; at least one death
```

In the worked example, times = [1, 2, 2, 3, 5], events = [1, 1, 0, 1, 0], t = 3 go through the equation above and give hazard = 0.2307692308, mean = 4.333333333, S = 0.5004199204. It applies only when one indicator per time; positive times; at least one death: outside that it says what is wrong instead of returning a number.

Kaplan-Meier product-limit survival estimate — `act_kaplan_meier`

The Kaplan-Meier estimate of the survival function at time t from observed times with death (1) or censoring (0) indicators; a life censored at a death time is counted at risk then.

$$\hat{S}(t) = \prod_{t_j \le t} \left(1 - \frac{d_j}{n_j}\right)$$

symbol		meaning	SI unit
`times`	in	observed times (a list)	—
`events`	in	1 for a death or failure, 0 for censoring (a list)	—
`t`	in	time at which the survival is wanted	—
`S`	out	Kaplan-Meier survival estimate S(t)	—
`F`	out	estimated probability of death by t	—
`deaths`	out	number of deaths observed up to t	—

Applies when: one indicator per time; positive times; indicators 0 or 1.

```
$ actuarial-science solve act_kaplan_meier "times=[1, 2, 2, 3, 5]" "events=[1, 1, 0, 1, 0]" t=3
S = 0.3
F = 0.7
deaths = 3
  by: Kaplan-Meier product-limit survival estimate  (S(t) = product over death times t_j <= t of (1
- d_j/n_j))
  t = 3
  assumes: one indicator per time; positive times; indicators 0 or 1
```

In the worked example, times = [1, 2, 2, 3, 5], events = [1, 1, 0, 1, 0], t = 3 go through the equation above and give S = 0.3, F = 0.7, deaths = 3. It applies only when one indicator per time; positive times; indicators 0 or 1: outside that it says what is wrong instead of returning a number.

Nelson-Aalen cumulative hazard estimate — `act_nelson_aalen`

The Nelson-Aalen estimate of the cumulative hazard at time t from observed times with death (1) or censoring (0) indicators, and the survival estimate exp(-H(t)).

$$\hat{H}(t) = \sum_{t_j \le t} \frac{d_j}{n_j}, \quad \hat{S}(t) = e^{-\hat{H}(t)}$$

symbol		meaning	SI unit
`times`	in	observed times (a list)	—
`events`	in	1 for a death or failure, 0 for censoring (a list)	—

symbol		meaning	SI unit
`t`	in	time at which the estimate is wanted	—
`H`	out	Nelson-Aalen cumulative hazard H(t)	—
`S`	out	survival estimate exp(-H(t))	—

Applies when: one indicator per time; positive times; indicators 0 or 1.

```
$ actuarial-science solve act_nelson_aalen "times=[1, 2, 2, 3, 5]" "events=[1, 1, 0, 1, 0]" t=3
H = 0.95
S = 0.3867410235
  by: Nelson-Aalen cumulative hazard estimate  (H(t) = sum over death times t_j <= t of d_j/n_j)
  t = 3
  assumes: one indicator per time; positive times; indicators 0 or 1
```

In the worked example, times = [1, 2, 2, 3, 5], events = [1, 1, 0, 1, 0], t = 3 go through the equation above and give H = 0.95, S = 0.3867410235. It applies only when one indicator per time; positive times; indicators 0 or 1: outside that it says what is wrong instead of returning a number.

Survival models (4)

Constant force of mortality (exponential lifetime) — `act_constant_force_mortality`

Survival and death probabilities, the complete and curtate expectations of life and the variance of the future lifetime when the force of mortality mu is constant.

$$ {}_tp_x = e^{-\mu t}, \quad \mathring{e}_x = \frac{1}{\mu}, \quad e_x = \frac{e^{-\mu}}{1 - e^{-\mu}} $$

symbol		meaning	SI unit
`mu`	in	constant force of mortality per year	—
`t`	in	number of years	—
`tpx`	out	probability of surviving t years	—
`tqx`	out	probability of dying within t years	—
`e_complete`	out	complete expectation 1/mu	—
`e_curtate`	out	curtate expectation p/(1-p)	—
`var_T`	out	variance of the future lifetime 1/mu^2	—

Applies when: 0 < mu < 5; t >= 0.

```
$ actuarial-science solve act_constant_force_mortality mu=0.02 t=10
tpx = 0.8187307531
tqx = 0.1812692469
e_complete = 50
e_curtate = 49.50166666
var_T = 2500
  by: Constant force of mortality (exponential lifetime)  (t_p_x = e^(-mu t); e° = 1/mu; e_x = e^(-
mu)/(1 - e^(-mu)))
  mu = 0.02
  t = 10
  assumes: 0 < mu < 5; t >= 0
```

In the worked example, mu = 0.02, t = 10 go through the equation above and give tpx = 0.8187307531, tqx = 0.1812692469, e_complete = 50, e_curtate = 49.50166666. It applies only when 0 < mu < 5; t >= 0: outside that it says what is wrong instead of returning a number.

De Moivre's law of mortality — `act_de_moivre_law`

Survival and death probabilities, the force of mortality, the complete expectation of life and the variance of the future lifetime under de Moivre's law (uniform future lifetime up to the limiting age omega).

$$ {}_tp_x = \frac{\omega - x - t}{\omega - x},\ \mu_{x+t} = \frac{1}{\omega - x - t},\ \mathring{e}_x = \frac{\omega - x}{2} $$

symbol		meaning	SI unit
`omega`	in	limiting age omega (years)	—
`x`	in	age of the life (years)	—
`t`	in	number of years	—
`tpx`	out	probability of surviving t years	—
`tqx`	out	probability of dying within t years	—
`mu`	out	force of mortality at age x+t	—
`e_complete`	out	complete expectation of life e°_x	—
`var_T`	out	variance of the future lifetime	—

Applies when: 0 <= x < omega; 0 <= t < omega - x.

```
$ actuarial-science solve act_de_moivre_law omega=100 x=40 t=10
tpx = 0.8333333333
tqx = 0.1666666667
mu = 0.02
e_complete = 30
var_T = 300
  by: De Moivre's law of mortality  (t_p_x = (omega - x - t)/(omega - x); mu_(x+t) = 1/(omega - x -
t); e°_x = (omega - x)/2; Var T = (omega - x)^2/12)
  omega = 100
  x = 40
  t = 10
  assumes: 0 <= x < omega; 0 <= t < omega - x
```

In the worked example, omega = 100, x = 40, t = 10 go through the equation above and give tpx = 0.8333333333, tqx = 0.1666666667, mu = 0.02, e_complete = 30. It applies only when 0 <= x < omega; 0 <= t < omega - x: outside that it says what is wrong instead of returning a number.

Gompertz's law of mortality — `act_gompertz_law`

Survival and death probabilities, the force of mortality at x and x+t and the complete expectation of life under Gompertz's law mu_x = B c^x.

$$\mu_x = Bc^x, \quad {}_tp_x = \exp\left(-\frac{Bc^x(c^t-1)}{\ln c}\right)$$

symbol		meaning	SI unit
`B`	in	Gompertz B (per year)	—
`c`	in	Gompertz c (> 1)	—
`x`	in	age of the life (years)	—
`t`	in	number of years	—
`tpx`	out	probability of surviving t years	—
`tqx`	out	probability of dying within t years	—
`mu_x`	out	force of mortality at age x	—
`mu_xt`	out	force of mortality at age x+t	—
`e_complete`	out	complete expectation of life e°_x	—

Applies when: 0 < B < 0.1; 1 < c < 1.5; 0 <= x <= 120; 0 <= t <= 100.

```
$ actuarial-science solve act_gompertz_law B=0.0003 c=1.07 x=50 t=10
tpx = 0.8813304297
tqx = 0.1186695703
mu_x = 0.008837107519
mu_xt = 0.01738392805
e_complete = 26.69114361
  by: Gompertz's law of mortality  (t_p_x = exp(-B c^x (c^t - 1)/ln c); e°_x = integral of t_p_x
(Gauss-Legendre))
  B = 0.0003
  c = 1.07
  x = 50
  t = 10
  assumes: 0 < B < 0.1; 1 < c < 1.5; 0 <= x <= 120; 0 <= t <= 100
```

In the worked example, B = 0.0003, c = 1.07, x = 50, t = 10 go through the equation above and give tpx = 0.8813304297, tqx = 0.1186695703, mu_x = 0.008837107519, mu_xt = 0.01738392805. It applies only when 0 < B < 0.1; 1 < c < 1.5; 0 <= x <= 120; 0 <= t <= 100: outside that it says what is wrong instead of returning a number.

Makeham's law of mortality — `act_makeham_law`

Survival and death probabilities, the force of mortality at x and x+t and the complete expectation of life under Makeham's law mu_x = A + B c^x.

$$\mu_x = A + Bc^x, \quad {}_tp_x = \exp\left(-At - \frac{Bc^x(c^t-1)}{\ln c}\right), \quad \mathring{e}_x = \int_0^\infty {}_tp_x\,dt$$

symbol		meaning	SI unit
`A`	in	Makeham's age-independent term A (per year)	—
`B`	in	Makeham's B (per year)	—
`c`	in	Makeham's c (> 1)	—
`x`	in	age of the life (years)	—
`t`	in	number of years	—
`tpx`	out	probability of surviving t years	—
`tqx`	out	probability of dying within t years	—
`mu_x`	out	force of mortality at age x	—
`mu_xt`	out	force of mortality at age x+t	—
`e_complete`	out	complete expectation of life e°_x	—

Applies when: 0 <= A < 0.1; 0 < B < 0.1; 1 < c < 1.5; 0 <= x <= 120; 0 <= t <= 100.

```
$ actuarial-science solve act_makeham_law A=0.00022 B=2.7e-06 c=1.124 x=60 t=10
tpx = 0.942549208
tqx = 0.05745079201
mu_x = 0.00322152827
mu_xt = 0.009880632256
e_complete = 27.20968666
  by: Makeham's law of mortality  (t_p_x = exp(-A t - B c^x (c^t - 1)/ln c); e°_x = integral of
t_p_x (Gauss-Legendre))
  A = 0.00022
  B = 2.7e-06
  c = 1.124
  x = 60
  t = 10
  assumes: 0 <= A < 0.1; 0 < B < 0.1; 1 < c < 1.5; 0 <= x <= 120; 0 <= t <= 100
```

In the worked example, A = 0.00022, B = 2.7e-06, c = 1.124, x = 60, t = 10 go through the equation above and give tpx = 0.942549208, tqx = 0.05745079201, mu_x = 0.00322152827, mu_xt = 0.009880632256. It applies only when 0 <= A < 0.1; 0 < B < 0.1; 1 < c < 1.5; 0 <= x <= 120; 0 <= t <= 100: outside that it says what is wrong instead of returning a number.

Questions and reports: support@ai999x.com. AI999X — The Future Is Self-Improving.